asset-class-performance-fed-policy-cycles

February 24, 2026

0.1 Introduction

In this post, we will look into the Fed Funds cycles and evaluate asset class performance during tightening and easing of monetary policy.

0.2 Python Imports

[1]:
```
# Standard Library
import datetime
import os
import sys
import warnings

from datetime import datetime
from pathlib import Path

# Data Handling
import numpy as np
import pandas as pd

# Data Sources
import pandas_datareader.data as web

# Statistical Analysis
import statsmodels.api as sm

# Machine Learning
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# Suppress warnings
warnings.filterwarnings("ignore")
```

0.3 Add Directories To Path

[2]:
```
# Add the source subdirectory to the system path to allow import config from␣
↪settings.py
current_directory = Path(os.getcwd())
```

```
website_base_directory = current_directory.parent.parent.parent
src_directory = website_base_directory / "src"
sys.path.append(str(src_directory)) if str(src_directory) not in sys.path else␣
↪None

# Import settings.py
from settings import config

# Add configured directories from config to path
SOURCE_DIR = config("SOURCE_DIR")
sys.path.append(str(Path(SOURCE_DIR))) if str(Path(SOURCE_DIR)) not in sys.path␣
↪else None

# Add other configured directories
BASE_DIR = config("BASE_DIR")
CONTENT_DIR = config("CONTENT_DIR")
POSTS_DIR = config("POSTS_DIR")
PAGES_DIR = config("PAGES_DIR")
PUBLIC_DIR = config("PUBLIC_DIR")
SOURCE_DIR = config("SOURCE_DIR")
DATA_DIR = config("DATA_DIR")
DATA_MANUAL_DIR = config("DATA_MANUAL_DIR")
```

0.4 Python Functions

Here are the functions needed for this project:

- bb_clean_data: Takes an Excel export from Bloomberg, removes the miscellaneous headings/rows, and returns a DataFrame.
- calc_fed_cycle_asset_performance: Calculates metrics for an asset based on a specified Fed tightening/loosening cycle.
- load_data: Load data from a CSV, Excel, or Pickle file into a pandas DataFrame.
- pandas_set_decimal_places: Set the number of decimal places displayed for floating-point numbers in pandas.
- plot_bar_returns_ffr_change: Plot the bar chart of the cumulative or annualized returns for the asset class along with the change in the Fed Funds Rate.
- plot_scatter_regression_ffr_vs_returns: Plot the scatter plot and regression of the annualized return for the asset class along with the annualized change in the Fed Funds Rate.
- plot_timeseries: Plot the timeseries data from a DataFrame for a specified date range and columns.
- summary_stats: Generate summary statistics for a series of returns.

[3]:
```
from bb_clean_data import bb_clean_data
from calc_fed_cycle_asset_performance import calc_fed_cycle_asset_performance
from load_data import load_data
from pandas_set_decimal_places import pandas_set_decimal_places
from plot_bar_returns_ffr_change import plot_bar_returns_ffr_change
```

```
from plot_scatter_regression_ffr_vs_returns import␣
↪plot_scatter_regression_ffr_vs_returns
from plot_timeseries import plot_timeseries
from summary_stats import summary_stats
```

0.5 Data Overview

0.5.1 Acquire & Plot Fed Funds Data

First, let's get the data for the Fed Funds target rate (FFR). This data is found in 3 different datasets from FRED.

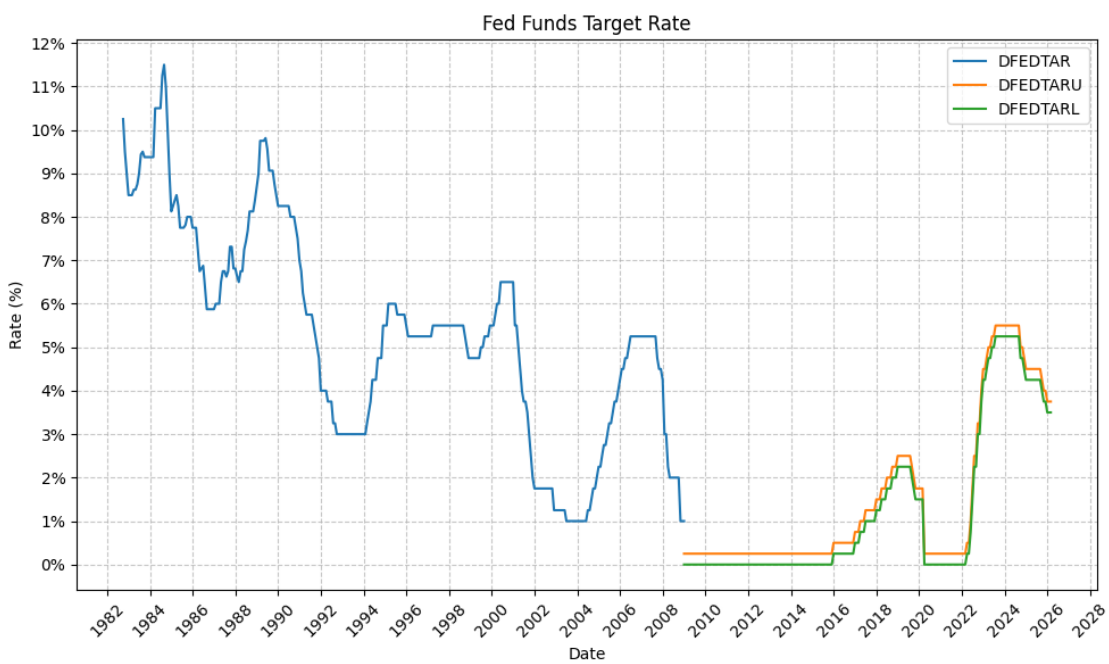
[4]:
```
# Set decimal places
pandas_set_decimal_places(5)

# Pull Federal Funds Target Rate (DISCONTINUED) (DFEDTAR)
fedfunds_target_old = web.DataReader("DFEDTAR", "fred", start="1900-01-01",␣
↪end=datetime.today())

# Pull Federal Funds Target Range - Upper Limit (DFEDTARU)
fedfunds_target_new_upper = web.DataReader("DFEDTARU", "fred",␣
↪start="1900-01-01", end=datetime.today())

# Pull Federal Funds Target Range - Lower Limit (DFEDTARL)
fedfunds_target_new_lower = web.DataReader("DFEDTARL", "fred",␣
↪start="1900-01-01", end=datetime.today())

# Merge the datasets together
fedfunds_combined = pd.concat([fedfunds_target_old, fedfunds_target_new_upper,␣
↪fedfunds_target_new_lower], axis=1)

# Divide all values by 100 to get decimal rates
for col in fedfunds_combined.columns:
    fedfunds_combined[col] = fedfunds_combined[col] / 100

# Resample to month-end (as if we know the rate at the end of the month)
fedfunds_monthly = fedfunds_combined.resample("ME").last()

display(fedfunds_monthly)
```

```
            DFEDTAR  DFEDTARU  DFEDTARL
DATE
1982-09-30  0.10250       NaN       NaN
1982-10-31  0.09500       NaN       NaN
1982-11-30  0.09000       NaN       NaN
1982-12-31  0.08500       NaN       NaN
1983-01-31  0.08500       NaN       NaN
…              …         …         …
```

```
2025-10-31       NaN   0.04000   0.03750
2025-11-30       NaN   0.04000   0.03750
2025-12-31       NaN   0.03750   0.03500
2026-01-31       NaN   0.03750   0.03500
2026-02-28       NaN   0.03750   0.03500

[522 rows x 3 columns]
```

We can then generate several useful plots. First, the Fed Funds target rate:

[5]:
```
plot_timeseries(
    price_df=fedfunds_monthly,
    plot_start_date=None,
    plot_end_date=None,
    plot_columns=["DFEDTAR", "DFEDTARU", "DFEDTARL"],
    title="Fed Funds Target Rate",
    x_label="Date",
    x_format="Year",
    x_tick_spacing=2,
    x_tick_rotation=45,
    y_label="Rate (%)",
    y_format="Percentage",
    y_format_decimal_places=0,
    y_tick_spacing=0.01,
    grid=True,
    legend=True,
    export_plot=False,
    plot_file_name=None,
)
```



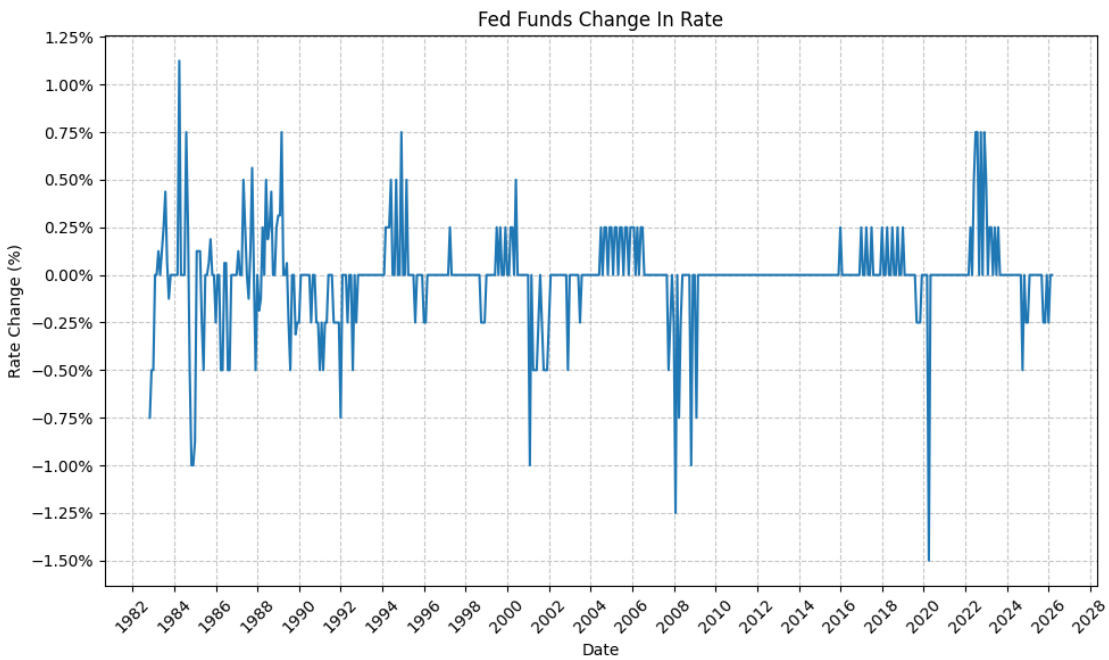
[6]:
```
# First drop the lower column and merge
fedfunds_monthly["fed_funds"] = fedfunds_monthly["DFEDTAR"].
 ↪combine_first(fedfunds_monthly["DFEDTARU"])
fedfunds_monthly = fedfunds_monthly.drop(columns=["DFEDTARL", "DFEDTARU",␣
 ↪"DFEDTAR"])

# Compute change in rate from either column
fedfunds_monthly["fed_funds_change"] = fedfunds_monthly["fed_funds"].diff()

display(fedfunds_monthly)
```

```
            fed_funds  fed_funds_change
DATE
1982-09-30    0.10250               NaN
1982-10-31    0.09500          -0.00750
1982-11-30    0.09000          -0.00500
1982-12-31    0.08500          -0.00500
1983-01-31    0.08500           0.00000
…               …               …
2025-10-31    0.04000          -0.00250
2025-11-30    0.04000           0.00000
2025-12-31    0.03750          -0.00250
2026-01-31    0.03750           0.00000
2026-02-28    0.03750           0.00000

[522 rows x 2 columns]
```

And then the change in FFR from month-to-month:

[7]:
```
plot_timeseries(
    price_df=fedfunds_monthly,
    plot_start_date=None,
    plot_end_date=None,
    plot_columns=["fed_funds_change"],
    title="Fed Funds Change In Rate",
    x_label="Date",
    x_format="Year",
    x_tick_spacing=2,
    x_tick_rotation=45,
    y_label="Rate Change (%)",
    y_format="Percentage",
    y_format_decimal_places=2,
    y_tick_spacing=0.0025,
    grid=True,
    legend=False,
```

```
    export_plot=False,
    plot_file_name=None,
)
```



This plot, in particular, makes it easy to show the monthly increase and decrease in the FFR, as well as the magnitude of the change (i.e. slow, drawn-out increases or decreases or abrupt large increases or decreases).

0.5.2 Define Fed Policy Cycles

Next, we will define the Fed policy tightening and easing cycles:

[8]:
```
# Set timeframe
start_date = "1989-12-15"
end_date = "2026-01-31"

# Copy DataFrame to avoid modifying original
fedfunds_cycles = fedfunds_monthly.copy()
fedfunds_cycles = fedfunds_cycles.loc[start_date:end_date]

# Reset index
fedfunds_cycles = fedfunds_cycles.reset_index()

# Drop the "fed_funds_change" column as we will use it to determine the cycle␣
↪type later, but we don't need it in the final dataframe
fedfunds_cycles = fedfunds_cycles.drop(columns=["fed_funds_change"])
```

```python
# Rename the date column to "Start Date", and the "fed_funds" column to "Fed␣
↪Funds Start"
fedfunds_cycles = fedfunds_cycles.rename(columns={"DATE": "start_date",␣
↪"fed_funds": "fed_funds_start"})

# Copy the "Start Date" column to a new column called "End Date" and shift it␣
↪up by one row
fedfunds_cycles["end_date"] = fedfunds_cycles["start_date"].shift(-1)

# Copy the "Fed Funds Start" column to a new column called "Fed Funds End" and␣
↪shift it up by one row
fedfunds_cycles["fed_funds_end"] = fedfunds_cycles["fed_funds_start"].shift(-1)

# Calculate the change in Fed Funds rate for each cycle
fedfunds_cycles["fed_funds_change"] = fedfunds_cycles["fed_funds_end"] -␣
↪fedfunds_cycles["fed_funds_start"]

# Based on "Fed Funds Change" column, determine if the rate is increasing,␣
↪decreasing, or unchanged
fedfunds_cycles["cycle"] = fedfunds_cycles["fed_funds_change"].apply(
    lambda x: "Tightening" if x > 0 else ("Easing" if x < 0 else "Neutral"))

display(fedfunds_cycles)
```

```
    start_date  fed_funds_start   end_date  fed_funds_end  fed_funds_change  \
0   1989-12-31          0.08250 1990-01-31        0.08250           0.00000
1   1990-01-31          0.08250 1990-02-28        0.08250           0.00000
2   1990-02-28          0.08250 1990-03-31        0.08250           0.00000
3   1990-03-31          0.08250 1990-04-30        0.08250           0.00000
4   1990-04-30          0.08250 1990-05-31        0.08250           0.00000
..         …                …        …             …                 …
429 2025-09-30          0.04250 2025-10-31        0.04000          -0.00250
430 2025-10-31          0.04000 2025-11-30        0.04000           0.00000
431 2025-11-30          0.04000 2025-12-31        0.03750          -0.00250
432 2025-12-31          0.03750 2026-01-31        0.03750           0.00000
433 2026-01-31          0.03750        NaT            NaN               NaN

       cycle
0    Neutral
1    Neutral
2    Neutral
3    Neutral
4    Neutral
..       …
429   Easing
430  Neutral
```

```
431   Easing
432  Neutral
433  Neutral

[434 rows x 6 columns]
```

Grouping the consecutive months of tightening and easing together, we can create a DataFrame that gives us the cumulative change in the FFR for each cycle, as well as the start and end dates for each cycle. This will be useful for evaluating asset class performance during these cycles.

[9]:
```
fedfunds_grouped_cycles = fedfunds_cycles.copy()

# Create a group key that increments whenever the Cycle value changes
fedfunds_grouped_cycles['group'] = (fedfunds_grouped_cycles['cycle'] !=␣
 ↪fedfunds_grouped_cycles['cycle'].shift(1)).cumsum()

# Group by both the group key and Cycle label
cycle_ranges = (
    fedfunds_grouped_cycles.groupby(['group', 'cycle'], sort=False)
    .agg(
        start_date=('start_date', 'first'),
        end_date=('end_date', 'last'),
        fed_funds_start=('fed_funds_start', 'first'),
        fed_funds_end=('fed_funds_end', 'last')
    )
    .reset_index(drop=False)
    .drop(columns='group')
)

display(cycle_ranges)
```

```
       cycle start_date   end_date  fed_funds_start  fed_funds_end
0    Neutral 1989-12-31 1990-06-30          0.08250        0.08250
1     Easing 1990-06-30 1990-07-31          0.08250        0.08000
2    Neutral 1990-07-31 1990-09-30          0.08000        0.08000
3     Easing 1990-09-30 1991-04-30          0.08000        0.05750
4    Neutral 1991-04-30 1991-07-31          0.05750        0.05750
..       …        …        …               …              …
116  Neutral 2024-12-31 2025-08-31          0.04500        0.04500
117   Easing 2025-08-31 2025-10-31          0.04500        0.04000
118  Neutral 2025-10-31 2025-11-30          0.04000        0.04000
119   Easing 2025-11-30 2025-12-31          0.04000        0.03750
120  Neutral 2025-12-31 2026-01-31          0.03750        0.03750

[121 rows x 5 columns]
```

Furthermore, we will make the assumption that any "Neutral" months (i.e. months where the FFR did not change) are part of the preceding cycle – to a point. For example, if we have a tightening month followed by a neutral month, we will consider the neutral month to be part of the

tightening cycle. This is a simplifying assumption, but it allows us to categorize all months into either tightening or easing cycles without having to create a separate category for neutral months. From a practical standpoint, this also makes it easier to evaluate asset class performance during these cycles, as we can simply look at the performance during the tightening and easing cycles without having to worry about the neutral months. We will foward fill no more than 6 months of neutral months, however, as it would be unreasonable to assume that a month of neutral policy is part of the preceding cycle if it has been 6 months since the last change in policy.

[10]:
```
fedfunds_grouped_cycles = fedfunds_cycles.copy()

# Forward fill cycle labels preceding "Neutral" cycles
fedfunds_grouped_cycles['cycle_filled'] = fedfunds_grouped_cycles['cycle'].
 ↪replace('Neutral', pd.NA).ffill(limit=6)

# Replaced any remaining <NA> values with "Modified Tightening"
fedfunds_grouped_cycles['cycle_filled'] =␣
 ↪fedfunds_grouped_cycles['cycle_filled'].fillna('Modified Tightening')

# Create a group key that increments whenever the Cycle value changes
fedfunds_grouped_cycles['group'] = (fedfunds_grouped_cycles['cycle_filled'] !=␣
 ↪fedfunds_grouped_cycles['cycle_filled'].shift(1)).cumsum()

# Group by both the group key and Cycle label
cycle_ranges = (
    fedfunds_grouped_cycles.groupby(['group', 'cycle_filled'], sort=False)
    .agg(
        start_date=('start_date', 'first'),
        end_date=('end_date', 'last'),
        fed_funds_start=('fed_funds_start', 'first'),
        fed_funds_end=('fed_funds_end', 'last')
    )
    .reset_index(drop=False)
    .drop(columns='group')
)

# Calc change in Fed Funds rate for each cycle
cycle_ranges["fed_funds_change"] = cycle_ranges["fed_funds_end"] -␣
 ↪cycle_ranges["fed_funds_start"]

# Add cycle labels
cycle_labels = [f"Cycle {i+1}" for i in range(len(cycle_ranges))]

# Combine labels with cycle_ranges
cycle_ranges['cycle_label'] = cycle_labels

display(cycle_ranges)
```

```
          cycle_filled start_date   end_date  fed_funds_start  fed_funds_end  \
```

0	Modified Tightening	1989-12-31	1990-06-30	0.08250	0.08250
1	Easing	1990-06-30	1993-03-31	0.08250	0.03000
2	Modified Tightening	1993-03-31	1994-01-31	0.03000	0.03000
3	Tightening	1994-01-31	1995-06-30	0.03000	0.06000
4	Easing	1995-06-30	1996-07-31	0.06000	0.05250
5	Modified Tightening	1996-07-31	1997-02-28	0.05250	0.05250
6	Tightening	1997-02-28	1997-09-30	0.05250	0.05500
7	Modified Tightening	1997-09-30	1998-08-31	0.05500	0.05500
8	Easing	1998-08-31	1999-05-31	0.05500	0.04750
9	Tightening	1999-05-31	2000-11-30	0.04750	0.06500
10	Modified Tightening	2000-11-30	2000-12-31	0.06500	0.06500
11	Easing	2000-12-31	2002-06-30	0.06500	0.01750
12	Modified Tightening	2002-06-30	2002-10-31	0.01750	0.01750
13	Easing	2002-10-31	2003-12-31	0.01750	0.01000
14	Modified Tightening	2003-12-31	2004-05-31	0.01000	0.01000
15	Tightening	2004-05-31	2006-12-31	0.01000	0.05250
16	Modified Tightening	2006-12-31	2007-08-31	0.05250	0.05250
17	Easing	2007-08-31	2009-07-31	0.05250	0.00250
18	Modified Tightening	2009-07-31	2015-11-30	0.00250	0.00250
19	Tightening	2015-11-30	2016-06-30	0.00250	0.00500
20	Modified Tightening	2016-06-30	2016-11-30	0.00500	0.00500
21	Tightening	2016-11-30	2019-06-30	0.00500	0.02500
22	Modified Tightening	2019-06-30	2019-07-31	0.02500	0.02500
23	Easing	2019-07-31	2020-09-30	0.02500	0.00250
24	Modified Tightening	2020-09-30	2022-02-28	0.00250	0.00250
25	Tightening	2022-02-28	2024-01-31	0.00250	0.05500
26	Modified Tightening	2024-01-31	2024-08-31	0.05500	0.05500
27	Easing	2024-08-31	2025-06-30	0.05500	0.04500
28	Modified Tightening	2025-06-30	2025-08-31	0.04500	0.04500
29	Easing	2025-08-31	2026-01-31	0.04500	0.03750

	fed_funds_change	cycle_label
0	0.00000	Cycle 1
1	-0.05250	Cycle 2
2	0.00000	Cycle 3
3	0.03000	Cycle 4
4	-0.00750	Cycle 5
5	0.00000	Cycle 6
6	0.00250	Cycle 7
7	0.00000	Cycle 8
8	-0.00750	Cycle 9
9	0.01750	Cycle 10
10	0.00000	Cycle 11
11	-0.04750	Cycle 12
12	0.00000	Cycle 13
13	-0.00750	Cycle 14
14	0.00000	Cycle 15
15	0.04250	Cycle 16

```
16           0.00000    Cycle 17
17          -0.05000    Cycle 18
18           0.00000    Cycle 19
19           0.00250    Cycle 20
20           0.00000    Cycle 21
21           0.02000    Cycle 22
22           0.00000    Cycle 23
23          -0.02250    Cycle 24
24           0.00000    Cycle 25
25           0.05250    Cycle 26
26           0.00000    Cycle 27
27          -0.01000    Cycle 28
28           0.00000    Cycle 29
29          -0.00750    Cycle 30
```

0.6 Asset Class Performance By Fed Policy Cycle

Moving on, we will now look at the performance of four (4) different asset classes during each Fed cycle. We'll use the following Bloomberg indices:

- SPXT_S&P 500 Total Return Index (Stocks)
- SPBDU10T_S&P US Treasury Bond 7-10 Year Total Return Index (Bonds)
- LF98TRUU_Bloomberg US Corporate High Yield Total Return Index Value Unhedged USD (High Yield Bonds)
- XAU_Gold USD Spot (Gold)

0.6.1 Stocks

First, we will clean and load the data for the S&P 500 Total Return Index (SPXT):

[11]:
```
bb_clean_data(
    base_directory=DATA_DIR,
    fund_ticker_name="SPXT_S&P 500 Total Return Index",
    source="Bloomberg",
    asset_class="Indices",
    excel_export=True,
    pickle_export=True,
    output_confirmation=False,
)

spxt = load_data(
    base_directory=DATA_DIR,
    ticker="SPXT_S&P 500 Total Return Index_Clean",
    source="Bloomberg",
    asset_class="Indices",
    timeframe="Daily",
    file_format="pickle",
)
```

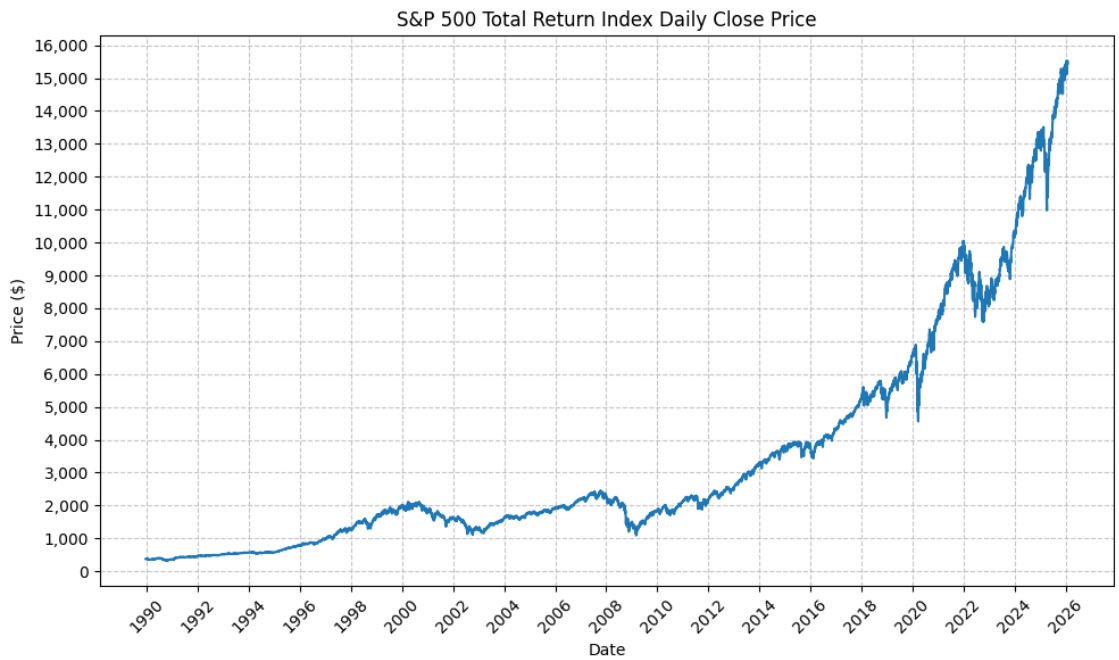
```
# Filter SPXT to date range
spxt = spxt[(spxt.index >= pd.to_datetime(start_date)) & (spxt.index <= pd.
 ↪to_datetime(end_date))]

# Drop everything except the "close" column
spxt = spxt[["Close"]]

# Resample to monthly frequency
spxt_monthly = spxt.resample("M").last()
spxt_monthly["Monthly_Return"] = spxt_monthly["Close"].pct_change()
spxt_monthly = spxt_monthly.dropna()

display(spxt_monthly)
```

```
                 Close  Monthly_Return
Date
1990-01-31   353.94000        -0.06713
1990-02-28   358.50000         0.01288
1990-03-31         368         0.02650
1990-04-30   358.81000        -0.02497
1990-05-31   393.80000         0.09752
…                 …             …
2025-09-30 14826.80000         0.03650
2025-10-31 15173.95000         0.02341
2025-11-30 15211.14000         0.00245
2025-12-31 15220.45000         0.00061
2026-01-31 15441.15000         0.01450

[433 rows x 2 columns]
```

Next, we can plot the price history before calculating the cycle performance:

[12]:
```
plot_timeseries(
    price_df=spxt,
    plot_start_date=None,
    plot_end_date=None,
    plot_columns=["Close"],
    title="S&P 500 Total Return Index Daily Close Price",
    x_label="Date",
    x_format="Year",
    x_tick_spacing=2,
    x_tick_rotation=45,
    y_label="Price ($)",
    y_format="Decimal",
    y_format_decimal_places=0,
    y_tick_spacing=1000,
    grid=True,
    legend=False,
```

```
    export_plot=False,
    plot_file_name=None,
)
```



Next, we will calculate the performance for SPY based on the pre-defined Fed cycles:

[13]:
```
spxt_cycles = calc_fed_cycle_asset_performance(
    start_date=cycle_ranges["start_date"],
    end_date=cycle_ranges["end_date"],
    label=cycle_ranges["cycle_label"],
    fed_funds_change=cycle_ranges["fed_funds_change"],
    monthly_returns=spxt_monthly,
)

display(spxt_cycles)
```

```
       Cycle       Start         End  Months  CumulativeReturn  \
0    Cycle 1  1989-12-31  1990-06-30       6           0.03092
1    Cycle 2  1990-06-30  1993-03-31      34           0.36800
2    Cycle 3  1993-03-31  1994-01-31      11           0.11359
3    Cycle 4  1994-01-31  1995-06-30      18           0.21800
4    Cycle 5  1995-06-30  1996-07-31      14           0.23230
5    Cycle 6  1996-07-31  1997-02-28       8           0.19592
6    Cycle 7  1997-02-28  1997-09-30       8           0.22017
7    Cycle 8  1997-09-30  1998-08-31      12           0.08094
8    Cycle 9  1998-08-31  1999-05-31      10           0.17553
```

9	Cycle 10	1999-05-31	2000-11-30	19	0.00401
10	Cycle 11	2000-11-30	2000-12-31	2	-0.07433
11	Cycle 12	2000-12-31	2002-06-30	19	-0.23106
12	Cycle 13	2002-06-30	2002-10-31	5	-0.16407
13	Cycle 14	2002-10-31	2003-12-31	15	0.39543
14	Cycle 15	2003-12-31	2004-05-31	6	0.06792
15	Cycle 16	2004-05-31	2006-12-31	32	0.34572
16	Cycle 17	2006-12-31	2007-08-31	9	0.06671
17	Cycle 18	2007-08-31	2009-07-31	24	-0.28840
18	Cycle 19	2009-07-31	2015-11-30	77	1.59039
19	Cycle 20	2015-11-30	2016-06-30	8	0.02502
20	Cycle 21	2016-06-30	2016-11-30	6	0.06008
21	Cycle 22	2016-11-30	2019-06-30	32	0.46031
22	Cycle 23	2019-06-30	2019-07-31	2	0.08586
23	Cycle 24	2019-07-31	2020-09-30	15	0.17105
24	Cycle 25	2020-09-30	2022-02-28	18	0.27728
25	Cycle 26	2022-02-28	2024-01-31	24	0.10892
26	Cycle 27	2024-01-31	2024-08-31	8	0.19526
27	Cycle 28	2024-08-31	2025-06-30	11	0.13779
28	Cycle 29	2025-06-30	2025-08-31	3	0.09622
29	Cycle 30	2025-08-31	2026-01-31	6	0.10133

	CumulativeReturnPct	AverageMonthlyReturn	AverageMonthlyReturnPct	\
0	3.09164	0.00634	0.63403	
1	36.80041	0.00996	0.99573	
2	11.35920	0.01001	1.00089	
3	21.80042	0.01141	1.14060	
4	23.23023	0.01527	1.52704	
5	19.59152	0.02336	2.33576	
6	22.01713	0.02631	2.63056	
7	8.09434	0.00812	0.81242	
8	17.55334	0.01849	1.84915	
9	0.40140	0.00127	0.12700	
10	-7.43308	-0.03697	-3.69725	
11	-23.10629	-0.01254	-1.25394	
12	-16.40672	-0.03266	-3.26564	
13	39.54292	0.02325	2.32521	
14	6.79152	0.01127	1.12720	
15	34.57166	0.00952	0.95192	
16	6.67103	0.00748	0.74817	
17	-28.83990	-0.01191	-1.19125	
18	159.03905	0.01314	1.31414	
19	2.50250	0.00356	0.35617	
20	6.00766	0.00997	0.99744	
21	46.03091	0.01256	1.25618	
22	8.58628	0.04242	4.24249	
23	17.10456	0.01234	1.23424	
24	27.72844	0.01457	1.45670	

25	10.89196	0.00584	0.58363
26	19.52588	0.02293	2.29282
27	13.77869	0.01244	1.24436
28	9.62171	0.03119	3.11890
29	10.13298	0.01629	1.62914

	AnnualizedReturn	AnnualizedReturnPct	Volatility	FedFundsChange	\
0	0.06279	6.27887	0.19170	0.00000	
1	0.11694	11.69426	0.13198	-0.05250	
2	0.12454	12.45375	0.06918	0.00000	
3	0.14051	14.05103	0.09928	0.03000	
4	0.19607	19.60729	0.07840	-0.00750	
5	0.30783	30.78278	0.14361	0.00000	
6	0.34782	34.78178	0.17547	0.00250	
7	0.08094	8.09434	0.20028	0.00000	
8	0.21418	21.41769	0.23543	-0.00750	
9	0.00253	0.25333	0.16483	0.01750	
10	-0.37088	-37.08781	0.20511	0.00000	
11	-0.15291	-15.29071	0.17313	-0.04750	
12	-0.34955	-34.95539	0.27616	0.00000	
13	0.30547	30.54681	0.14377	-0.00750	
14	0.14044	14.04429	0.08737	0.00000	
15	0.11778	11.77833	0.06998	0.04250	
16	0.08992	8.99217	0.08722	0.00000	
17	-0.15644	-15.64355	0.22888	-0.05000	
18	0.15990	15.99000	0.13125	0.00000	
19	0.03777	3.77714	0.11389	0.00250	
20	0.12376	12.37623	0.07708	0.00000	
21	0.15257	15.25686	0.12685	0.02000	
22	0.63927	63.92672	0.13743	0.00000	
23	0.13464	13.46425	0.21140	-0.02250	
24	0.17722	17.72221	0.15078	0.00000	
25	0.05305	5.30525	0.19480	0.05250	
26	0.30675	30.67513	0.10238	0.00000	
27	0.15122	15.12175	0.13032	-0.01000	
28	0.44406	44.40637	0.05911	0.00000	
29	0.21293	21.29273	0.04688	-0.00750	

	FedFundsChange_bps	FFR_AnnualizedChange	FFR_AnnualizedChange_bps	\
0	0.00000	0.00000	0.00000	
1	-525.00000	-0.01853	-185.29412	
2	0.00000	0.00000	0.00000	
3	300.00000	0.02000	200.00000	
4	-75.00000	-0.00643	-64.28571	
5	0.00000	0.00000	0.00000	
6	25.00000	0.00375	37.50000	
7	0.00000	0.00000	0.00000	
8	-75.00000	-0.00900	-90.00000	

9	175.00000	0.01105	110.52632
10	0.00000	0.00000	0.00000
11	-475.00000	-0.03000	-300.00000
12	0.00000	0.00000	0.00000
13	-75.00000	-0.00600	-60.00000
14	0.00000	0.00000	0.00000
15	425.00000	0.01594	159.37500
16	0.00000	0.00000	0.00000
17	-500.00000	-0.02500	-250.00000
18	0.00000	0.00000	0.00000
19	25.00000	0.00375	37.50000
20	0.00000	0.00000	0.00000
21	200.00000	0.00750	75.00000
22	0.00000	0.00000	0.00000
23	-225.00000	-0.01800	-180.00000
24	0.00000	0.00000	0.00000
25	525.00000	0.02625	262.50000
26	0.00000	0.00000	0.00000
27	-100.00000	-0.01091	-109.09091
28	0.00000	0.00000	0.00000
29	-75.00000	-0.01500	-150.00000

	Label
0	Cycle 1, 1989-12-31 to 1990-06-30
1	Cycle 2, 1990-06-30 to 1993-03-31
2	Cycle 3, 1993-03-31 to 1994-01-31
3	Cycle 4, 1994-01-31 to 1995-06-30
4	Cycle 5, 1995-06-30 to 1996-07-31
5	Cycle 6, 1996-07-31 to 1997-02-28
6	Cycle 7, 1997-02-28 to 1997-09-30
7	Cycle 8, 1997-09-30 to 1998-08-31
8	Cycle 9, 1998-08-31 to 1999-05-31
9	Cycle 10, 1999-05-31 to 2000-11-30
10	Cycle 11, 2000-11-30 to 2000-12-31
11	Cycle 12, 2000-12-31 to 2002-06-30
12	Cycle 13, 2002-06-30 to 2002-10-31
13	Cycle 14, 2002-10-31 to 2003-12-31
14	Cycle 15, 2003-12-31 to 2004-05-31
15	Cycle 16, 2004-05-31 to 2006-12-31
16	Cycle 17, 2006-12-31 to 2007-08-31
17	Cycle 18, 2007-08-31 to 2009-07-31
18	Cycle 19, 2009-07-31 to 2015-11-30
19	Cycle 20, 2015-11-30 to 2016-06-30
20	Cycle 21, 2016-06-30 to 2016-11-30
21	Cycle 22, 2016-11-30 to 2019-06-30
22	Cycle 23, 2019-06-30 to 2019-07-31
23	Cycle 24, 2019-07-31 to 2020-09-30
24	Cycle 25, 2020-09-30 to 2022-02-28

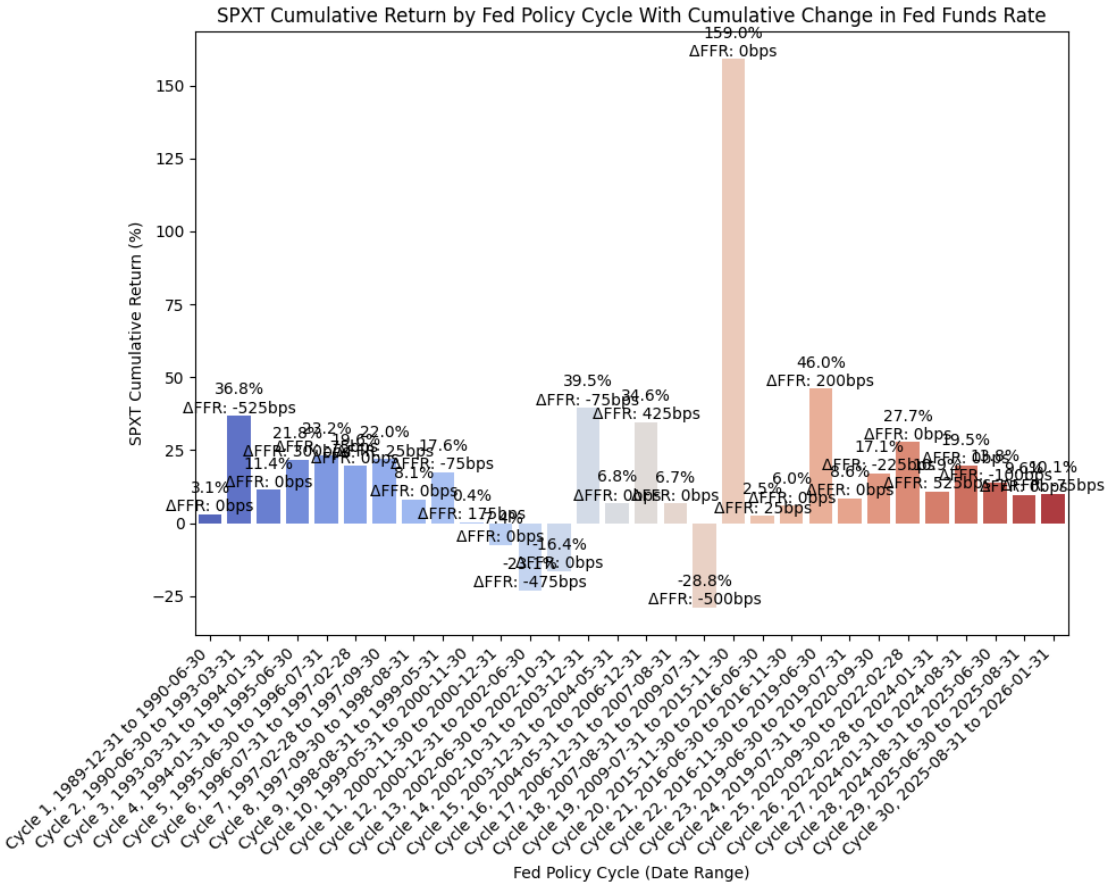
```
25  Cycle 26, 2022-02-28 to 2024-01-31
26  Cycle 27, 2024-01-31 to 2024-08-31
27  Cycle 28, 2024-08-31 to 2025-06-30
28  Cycle 29, 2025-06-30 to 2025-08-31
29  Cycle 30, 2025-08-31 to 2026-01-31
```

This gives us the following data points:

- Cycle start date
- Cycle end date
- Number of months in the cycle
- Cumulative return during the cycle (decimal and percent)
- Average monthly return during the cycle (decimal and percent)
- Annualized return during the cycle (decimal and percent)
- Return volatility during the cycle
- Cumulative change in FFR during the cycle (decimal and basis points)
- Annualized change in FFR during the cycle (decimal and basis points)

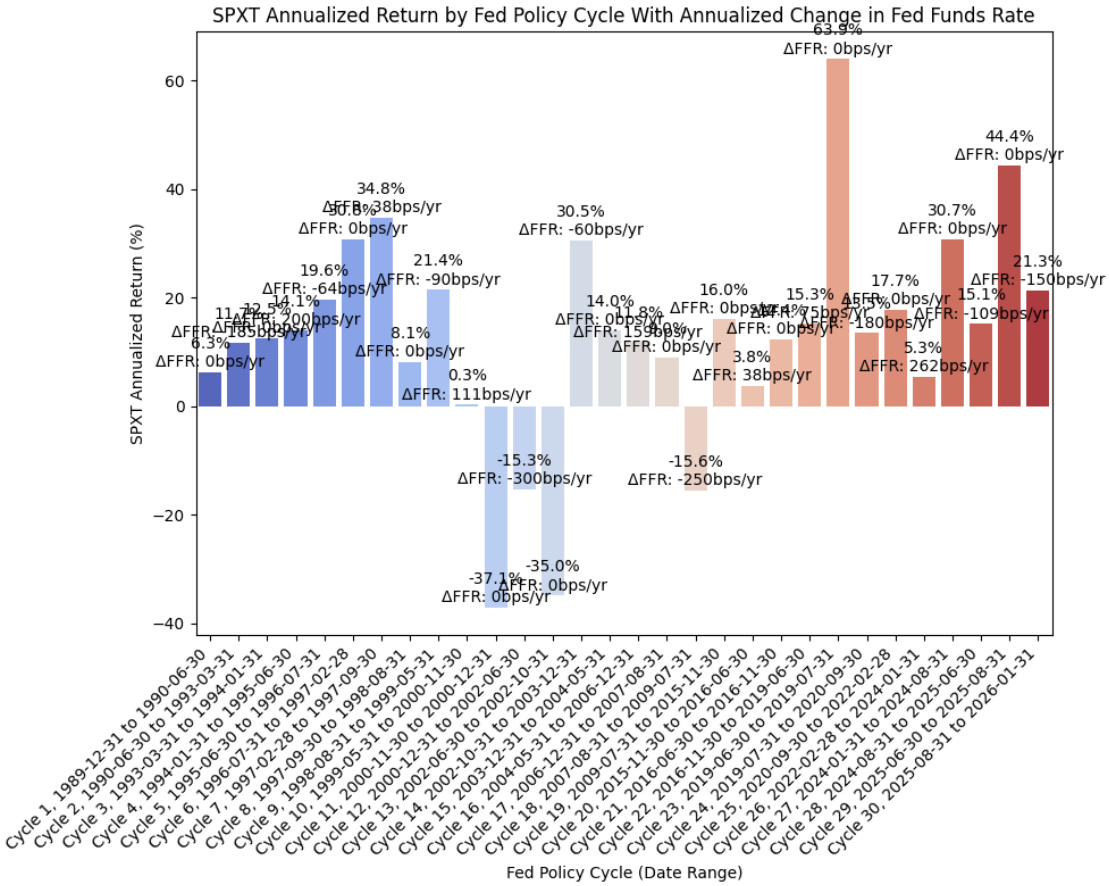
From the above DataFrame, we can then plot the cumulative and annualized returns for each cycle in a bar chart. First, the cumulative returns along with the cumulative change in FFR:

[14]:
```
plot_bar_returns_ffr_change(
    cycle_df=spxt_cycles,
    asset_label="SPXT",
    annualized_or_cumulative="Cumulative",
)
```



And then the annualized returns along with the annualized change in FFR:

[15]:
```
plot_bar_returns_ffr_change(
    cycle_df=spxt_cycles,
    asset_label="SPXT",
    annualized_or_cumulative="Annualized",
)
```



The cumulative returns plot is not particularly insightful, but there are some interesting observations to be gained from the annualized returns plot. During the past two (2) rate cutting cycles (cycles 11/12/13/14 and 18), stocks have exhibited negative returns during the rate cutting cycle. However, after the rate cutting cycle was complete, returns during the following cycle (when rates were usually flat) were quite strong and higher than the historical mean return for the S&P 500. The economic intuition for this behavior is valid; as the economy weakens, investors are concerned about the pricing of equities, the returns become negative, and the Fed responds with cutting rates. The exact timing of when the Fed begins cutting rates is one of the unknowns; the Fed could be ahead of the curve, cutting rates as economic data begins to prompt that action, or behind the curve, where the ecomony rolls over rapidly and even the Fed's actions are not enough to halt the economic contraction.

Finally, we can run an OLS regression to check fit:

[16]:
```
df = spxt_cycles

#=== Don't modify below this line ===

# Run OLS regression with statsmodels
X = df["FFR_AnnualizedChange_bps"]
```

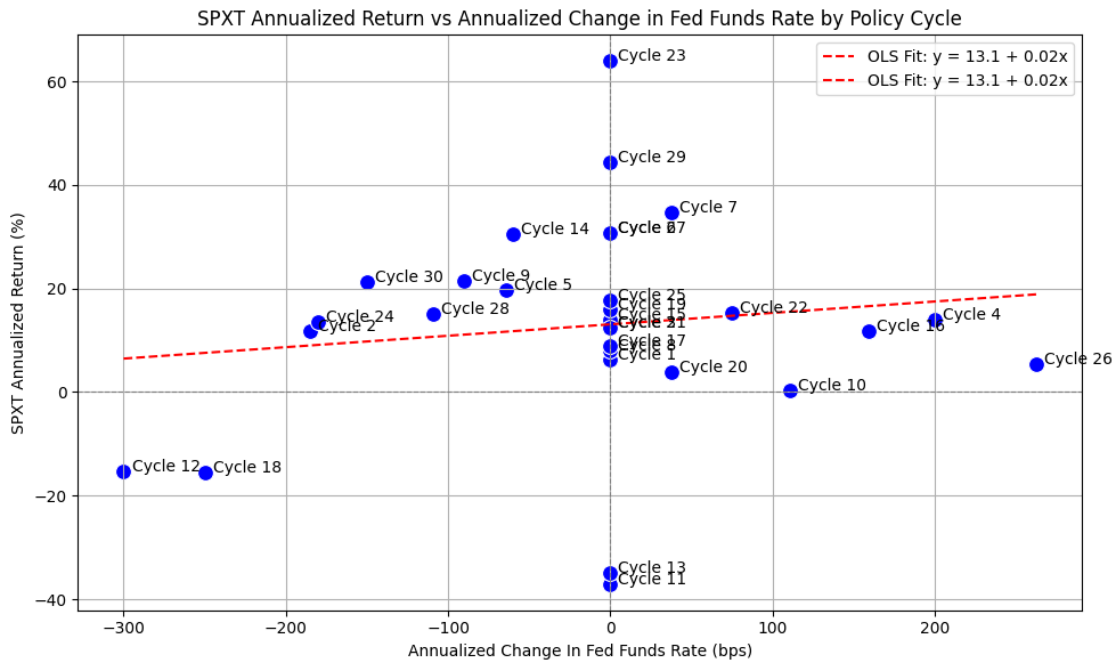
```
y = df["AnnualizedReturnPct"]
X = sm.add_constant(X)
model = sm.OLS(y, X).fit()
print(model.summary())
print(f"Intercept: {model.params[0]}, Slope: {model.params[1]}")  # Intercept␣
↪and slope

# Calc X and Y values for regression line
X_vals = np.linspace(X.min(), X.max(), 100)
Y_vals = model.params[0] + model.params[1] * X_vals
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:     AnnualizedReturnPct   R-squared:                       0.017
Model:                             OLS   Adj. R-squared:                 -0.018
Method:                  Least Squares   F-statistic:                    0.4786
Date:                 Tue, 24 Feb 2026   Prob (F-statistic):              0.495
Time:                         14:11:31   Log-Likelihood:                -132.24
No. Observations:                   30   AIC:                             268.5
Df Residuals:                       28   BIC:                             271.3
Df Model:                            1
Covariance Type:             nonrobust
================================================================================
============
                               coef    std err          t      P>|t|      [0.025
0.975]
--------------------------------------------------------------------------------
------------
const                       13.0760      3.792      3.448      0.002       5.308
20.844
FFR_AnnualizedChange_bps     0.0221      0.032      0.692      0.495      -0.043
0.087
==============================================================================
Omnibus:                        4.070   Durbin-Watson:                   1.523
Prob(Omnibus):                  0.131   Jarque-Bera (JB):                2.894
Skew:                          -0.320   Prob(JB):                        0.235
Kurtosis:                       4.381   Cond. No.                         120.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
Intercept: 13.076015539043272, Slope: 0.022056651501767076
```

And then plot the regression line along with the values:

[17]:
```
plot_scatter_regression_ffr_vs_returns(
    cycle_df=spxt_cycles,
```

```
    asset_label="SPXT",
    x_vals=X_vals,
    y_vals=Y_vals,
    intercept=model.params[0],
    slope=model.params[1],
)
```



Here we can see the data points for cycles 11/12/13/14 and 18 as mentioned above. Interestingly, cycles 28/29/30 (which is the current rate cutting cycle) appears to be an outlier. Of course, the book is not yet finished for cycles 28/29/30, and we could certainly see a bear market in stocks over the next several years.

0.6.2 Bonds

Next, we'll run a similar process for medium term bonds using SPBDU10T_S&P US Treasury Bond 7-10 Year Total Return Index.

First, we pull data with the following:

[18]:
```
bb_clean_data(
    base_directory=DATA_DIR,
    fund_ticker_name="SPBDU10T_S&P US Treasury Bond 7-10 Year Total Return␣
 ↪Index",
    source="Bloomberg",
    asset_class="Indices",
    excel_export=True,
```

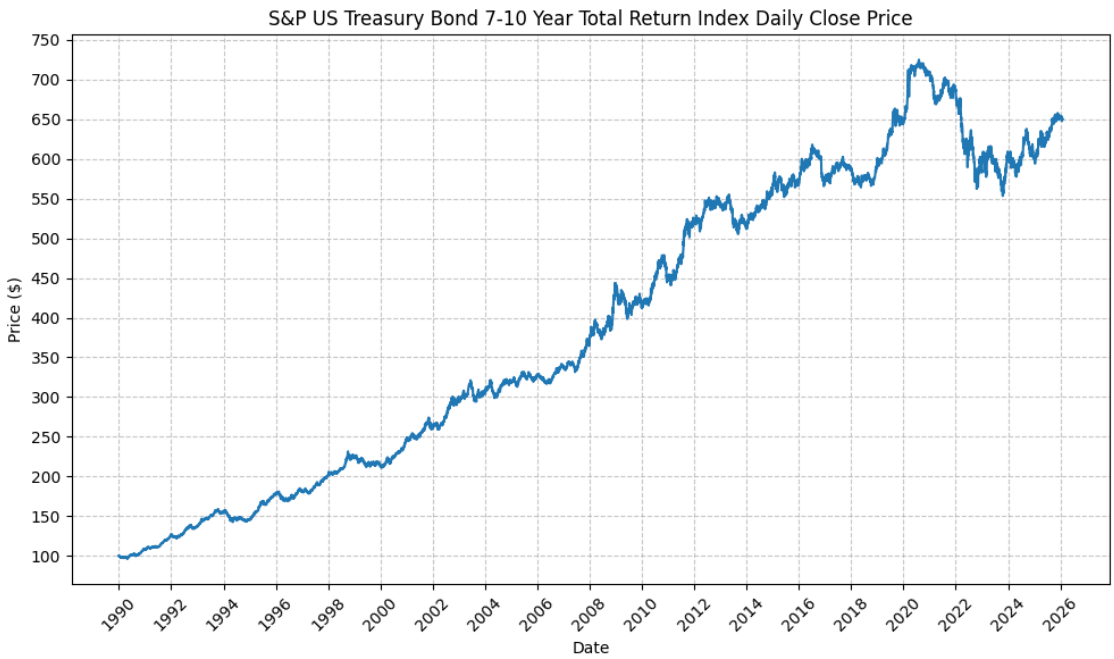
```
    pickle_export=True,
    output_confirmation=False,
)

treas_10y = load_data(
    base_directory=DATA_DIR,
    ticker="SPBDU10T_S&P US Treasury Bond 7-10 Year Total Return Index_Clean",
    source="Bloomberg",
    asset_class="Indices",
    timeframe="Daily",
    file_format="pickle",
)

# Filter TREAS_10Y to date range
treas_10y = treas_10y[(treas_10y.index >= pd.to_datetime(start_date)) &␣
 ↪(treas_10y.index <= pd.to_datetime(end_date))]

# Drop everything except the "close" column
treas_10y = treas_10y[["Close"]]

# Resample to monthly frequency
treas_10y_monthly = treas_10y.resample("M").last()
treas_10y_monthly["Monthly_Return"] = treas_10y_monthly["Close"].pct_change()
treas_10y_monthly = treas_10y_monthly.dropna()

display(treas_10y_monthly)
```

```
               Close  Monthly_Return
Date
1990-01-31  98.01300        -0.01987
1990-02-28  97.99000        -0.00023
1990-03-31  97.98900        -0.00001
1990-04-30  96.60600        -0.01411
1990-05-31  99.64700         0.03148
…               …             …
2025-09-30 645.58400         0.00675
2025-10-31 650.00500         0.00685
2025-11-30 656.63600         0.01020
2025-12-31 652.28800        -0.00662
2026-01-31 650.16900        -0.00325

[433 rows x 2 columns]
```

Next, we can plot the price history before calculating the cycle performance:

[19]:
```
plot_timeseries(
    price_df=treas_10y,
    plot_start_date=None,
```

```
    plot_end_date=None,
    plot_columns=["Close"],
    title="S&P US Treasury Bond 7-10 Year Total Return Index Daily Close Price",
    x_label="Date",
    x_format="Year",
    x_tick_spacing=2,
    x_tick_rotation=45,
    y_label="Price ($)",
    y_format="Decimal",
    y_format_decimal_places=0,
    y_tick_spacing=50,
    grid=True,
    legend=False,
    export_plot=False,
    plot_file_name=None,
)
```



Next, we will calculate the performance for SPY based on the pre-defined Fed cycles:

[20]:
```
treas_10y_cycles = calc_fed_cycle_asset_performance(
    start_date=cycle_ranges["start_date"],
    end_date=cycle_ranges["end_date"],
    label=cycle_ranges["cycle_label"],
    fed_funds_change=cycle_ranges["fed_funds_change"],
    monthly_returns=treas_10y_monthly,
)
```

```
display(treas_10y_cycles)
```

```
       Cycle       Start         End  Months  CumulativeReturn  \
0    Cycle 1  1989-12-31  1990-06-30       6           0.01362
1    Cycle 2  1990-06-30  1993-03-31      34           0.45682
2    Cycle 3  1993-03-31  1994-01-31      11           0.08876
3    Cycle 4  1994-01-31  1995-06-30      18           0.07796
4    Cycle 5  1995-06-30  1996-07-31      14           0.04785
5    Cycle 6  1996-07-31  1997-02-28       8           0.05301
6    Cycle 7  1997-02-28  1997-09-30       8           0.06453
7    Cycle 8  1997-09-30  1998-08-31      12           0.14428
8    Cycle 9  1998-08-31  1999-05-31      10           0.03052
9   Cycle 10  1999-05-31  2000-11-30      19           0.08744
10  Cycle 11  2000-11-30  2000-12-31       2           0.04918
11  Cycle 12  2000-12-31  2002-06-30      19           0.14331
12  Cycle 13  2002-06-30  2002-10-31       5           0.09945
13  Cycle 14  2002-10-31  2003-12-31      15           0.02161
14  Cycle 15  2003-12-31  2004-05-31       6          -0.00088
15  Cycle 16  2004-05-31  2006-12-31      32           0.10267
16  Cycle 17  2006-12-31  2007-08-31       9           0.03300
17  Cycle 18  2007-08-31  2009-07-31      24           0.19843
18  Cycle 19  2009-07-31  2015-11-30      77           0.38995
19  Cycle 20  2015-11-30  2016-06-30       8           0.06790
20  Cycle 21  2016-06-30  2016-11-30       6          -0.03175
21  Cycle 22  2016-11-30  2019-06-30      32           0.05923
22  Cycle 23  2019-06-30  2019-07-31       2           0.01306
23  Cycle 24  2019-07-31  2020-09-30      15           0.13083
24  Cycle 25  2020-09-30  2022-02-28      18          -0.06522
25  Cycle 26  2022-02-28  2024-01-31      24          -0.09878
26  Cycle 27  2024-01-31  2024-08-31       8           0.02748
27  Cycle 28  2024-08-31  2025-06-30      11           0.03294
28  Cycle 29  2025-06-30  2025-08-31       3           0.02670
29  Cycle 30  2025-08-31  2026-01-31       6           0.03081

    CumulativeReturnPct  AverageMonthlyReturn  AverageMonthlyReturnPct  \
0               1.36200               0.00241                  0.24101
1              45.68226               0.01124                  1.12428
2               8.87610               0.00784                  0.78356
3               7.79638               0.00439                  0.43870
4               4.78467               0.00344                  0.34387
5               5.30118               0.00658                  0.65819
6               6.45298               0.00799                  0.79904
7              14.42797               0.01135                  1.13516
8               3.05162               0.00326                  0.32630
9               8.74418               0.00449                  0.44926
10              4.91839               0.02430                  2.42972
11             14.33071               0.00721                  0.72113
```

12	9.94496	0.01929	1.92905
13	2.16130	0.00172	0.17218
14	-0.08831	0.00006	0.00600
15	10.26670	0.00314	0.31382
16	3.30047	0.00371	0.37102
17	19.84327	0.00792	0.79193
18	38.99485	0.00443	0.44302
19	6.78998	0.00835	0.83470
20	-3.17490	-0.00513	-0.51254
21	5.92338	0.00190	0.19002
22	1.30568	0.00653	0.65261
23	13.08268	0.00837	0.83706
24	-6.52177	-0.00366	-0.36622
25	-9.87840	-0.00394	-0.39369
26	2.74817	0.00357	0.35718
27	3.29408	0.00310	0.31010
28	2.66978	0.00888	0.88750
29	3.08068	0.00510	0.51007

	AnnualizedReturn	AnnualizedReturnPct	Volatility	FedFundsChange	\
0	0.02743	2.74255	0.06657	0.00000	
1	0.14202	14.20180	0.05346	-0.05250	
2	0.09721	9.72108	0.04458	0.00000	
3	0.05132	5.13229	0.07287	0.03000	
4	0.04087	4.08738	0.04940	-0.00750	
5	0.08056	8.05623	0.05365	0.00000	
6	0.09834	9.83398	0.06285	0.00250	
7	0.14428	14.42797	0.03891	0.00000	
8	0.03673	3.67303	0.08244	-0.00750	
9	0.05437	5.43706	0.04250	0.01750	
10	0.33386	33.38584	0.00458	0.00000	
11	0.08826	8.82645	0.05901	-0.04750	
12	0.25551	25.55118	0.06682	0.00000	
13	0.01725	1.72534	0.08696	-0.00750	
14	-0.00177	-0.17654	0.07653	0.00000	
15	0.03733	3.73293	0.04440	0.04250	
16	0.04425	4.42466	0.05098	0.00000	
17	0.09473	9.47295	0.09432	-0.05000	
18	0.05265	5.26537	0.05945	0.00000	
19	0.10356	10.35595	0.05317	0.00250	
20	-0.06249	-6.24899	0.08242	0.00000	
21	0.02181	2.18142	0.04975	0.02000	
22	0.08094	8.09429	0.03024	0.00000	
23	0.10336	10.33591	0.06056	-0.02250	
24	-0.04397	-4.39653	0.04430	0.00000	
25	-0.05068	-5.06760	0.09858	0.05250	
26	0.04150	4.15045	0.06957	0.00000	
27	0.03599	3.59887	0.06280	-0.01000	

28	0.11114	11.11445	0.04409	0.00000
29	0.06256	6.25626	0.02992	-0.00750

	FedFundsChange_bps	FFR_AnnualizedChange	FFR_AnnualizedChange_bps	\
0	0.00000	0.00000	0.00000	
1	-525.00000	-0.01853	-185.29412	
2	0.00000	0.00000	0.00000	
3	300.00000	0.02000	200.00000	
4	-75.00000	-0.00643	-64.28571	
5	0.00000	0.00000	0.00000	
6	25.00000	0.00375	37.50000	
7	0.00000	0.00000	0.00000	
8	-75.00000	-0.00900	-90.00000	
9	175.00000	0.01105	110.52632	
10	0.00000	0.00000	0.00000	
11	-475.00000	-0.03000	-300.00000	
12	0.00000	0.00000	0.00000	
13	-75.00000	-0.00600	-60.00000	
14	0.00000	0.00000	0.00000	
15	425.00000	0.01594	159.37500	
16	0.00000	0.00000	0.00000	
17	-500.00000	-0.02500	-250.00000	
18	0.00000	0.00000	0.00000	
19	25.00000	0.00375	37.50000	
20	0.00000	0.00000	0.00000	
21	200.00000	0.00750	75.00000	
22	0.00000	0.00000	0.00000	
23	-225.00000	-0.01800	-180.00000	
24	0.00000	0.00000	0.00000	
25	525.00000	0.02625	262.50000	
26	0.00000	0.00000	0.00000	
27	-100.00000	-0.01091	-109.09091	
28	0.00000	0.00000	0.00000	
29	-75.00000	-0.01500	-150.00000	

	Label
0	Cycle 1, 1989-12-31 to 1990-06-30
1	Cycle 2, 1990-06-30 to 1993-03-31
2	Cycle 3, 1993-03-31 to 1994-01-31
3	Cycle 4, 1994-01-31 to 1995-06-30
4	Cycle 5, 1995-06-30 to 1996-07-31
5	Cycle 6, 1996-07-31 to 1997-02-28
6	Cycle 7, 1997-02-28 to 1997-09-30
7	Cycle 8, 1997-09-30 to 1998-08-31
8	Cycle 9, 1998-08-31 to 1999-05-31
9	Cycle 10, 1999-05-31 to 2000-11-30
10	Cycle 11, 2000-11-30 to 2000-12-31
11	Cycle 12, 2000-12-31 to 2002-06-30

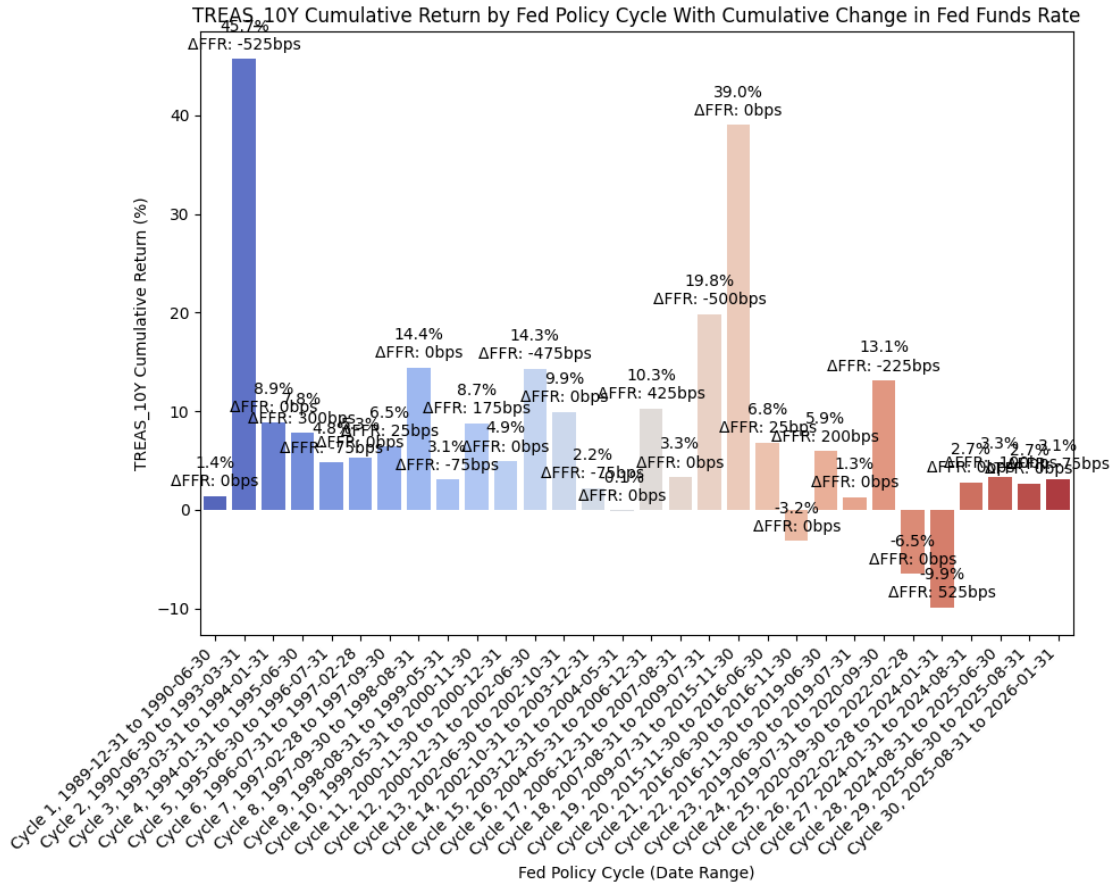
```
12  Cycle 13, 2002-06-30 to 2002-10-31
13  Cycle 14, 2002-10-31 to 2003-12-31
14  Cycle 15, 2003-12-31 to 2004-05-31
15  Cycle 16, 2004-05-31 to 2006-12-31
16  Cycle 17, 2006-12-31 to 2007-08-31
17  Cycle 18, 2007-08-31 to 2009-07-31
18  Cycle 19, 2009-07-31 to 2015-11-30
19  Cycle 20, 2015-11-30 to 2016-06-30
20  Cycle 21, 2016-06-30 to 2016-11-30
21  Cycle 22, 2016-11-30 to 2019-06-30
22  Cycle 23, 2019-06-30 to 2019-07-31
23  Cycle 24, 2019-07-31 to 2020-09-30
24  Cycle 25, 2020-09-30 to 2022-02-28
25  Cycle 26, 2022-02-28 to 2024-01-31
26  Cycle 27, 2024-01-31 to 2024-08-31
27  Cycle 28, 2024-08-31 to 2025-06-30
28  Cycle 29, 2025-06-30 to 2025-08-31
29  Cycle 30, 2025-08-31 to 2026-01-31
```

This gives us the following data points:

- Cycle start date
- Cycle end date
- Number of months in the cycle
- Cumulative return during the cycle (decimal and percent)
- Average monthly return during the cycle (decimal and percent)
- Annualized return during the cycle (decimal and percent)
- Return volatility during the cycle
- Cumulative change in FFR during the cycle (decimal and basis points)
- Annualized change in FFR during the cycle (decimal and basis points)

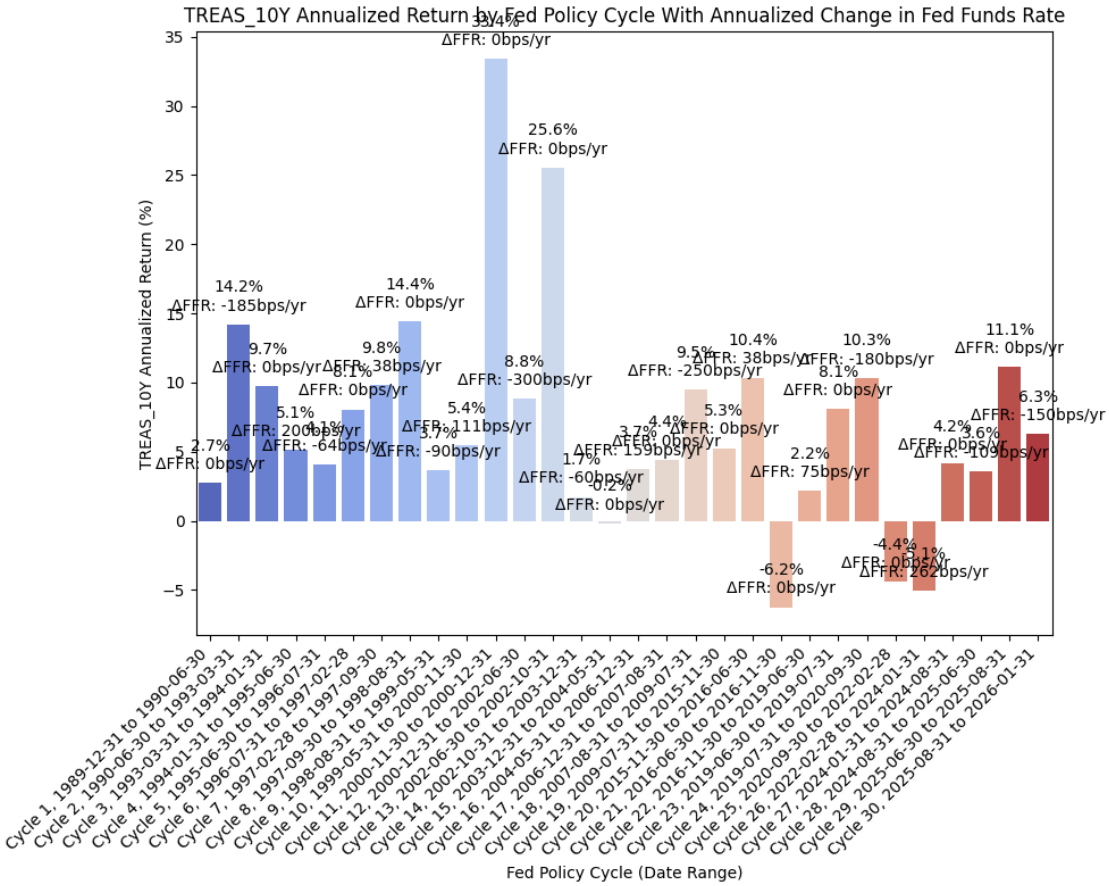
From the above DataFrame, we can then plot the cumulative and annualized returns for each cycle in a bar chart. First, the cumulative returns along with the cumulative change in FFR:

[21]:
```
plot_bar_returns_ffr_change(
    cycle_df=treas_10y_cycles,
    asset_label="TREAS_10Y",
    annualized_or_cumulative="Cumulative",
)
```



And then the annualized returns along with the annualized change in FFR:

[22]:
```
plot_bar_returns_ffr_change(
    cycle_df=treas_10y_cycles,
    asset_label="TREAS_10Y",
    annualized_or_cumulative="Annualized",
)
```



For this dataset, we have cycles 11/12/13/14 exhibiting strong returns, which is consistent with the economic intuition that bonds should perform well during periods of economic weakness and rate cuts. We also see this outcome with cycle 18. It becomes a little more interesting during cycles 25 and 26, where the correlations of stocks and bond returns seemed to align, so we see negative bond returns there. Finally, cycles 28/29/30 also exhibit positive bond returns, which is consistent with our thesis that bonds should perform well during periods of economic weakness and rate cuts.

Finally, we can run an OLS regression to check fit:

[23]:
```
df = treas_10y_cycles

#=== Don't modify below this line ===

# Run OLS regression with statsmodels
X = df["FFR_AnnualizedChange_bps"]
y = df["AnnualizedReturnPct"]
X = sm.add_constant(X)
model = sm.OLS(y, X).fit()
print(model.summary())
```

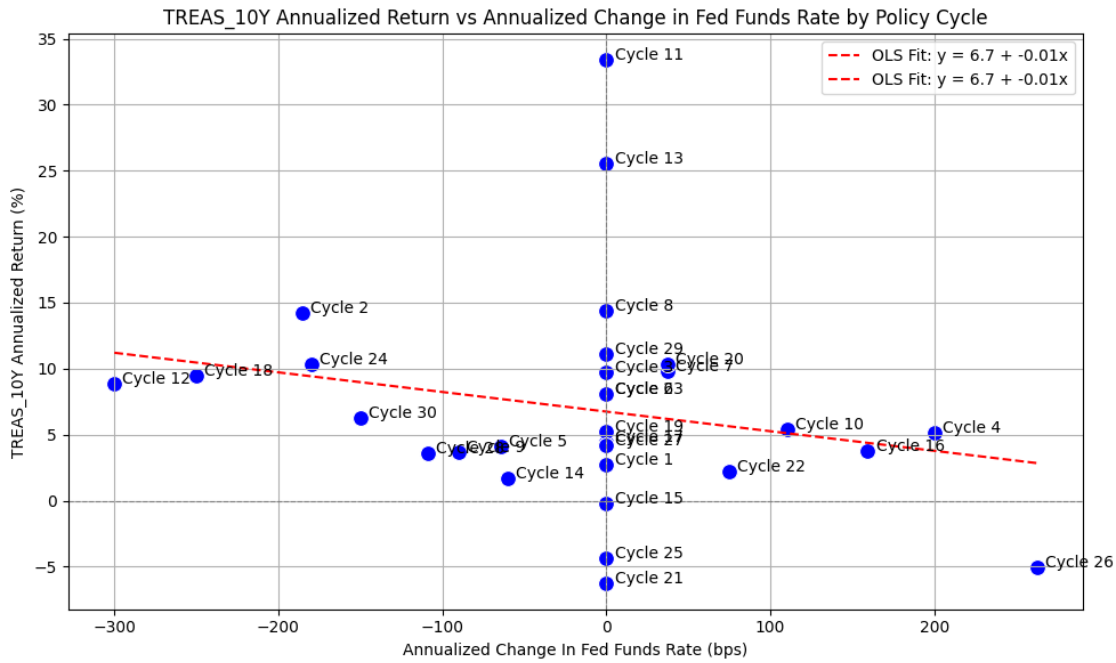
```
print(f"Intercept: {model.params[0]}, Slope: {model.params[1]}")  # Intercept␣
↪and slope

# Calc X and Y values for regression line
X_vals = np.linspace(X.min(), X.max(), 100)
Y_vals = model.params[0] + model.params[1] * X_vals
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:     AnnualizedReturnPct   R-squared:                       0.050
Model:                             OLS   Adj. R-squared:                  0.016
Method:                  Least Squares   F-statistic:                     1.462
Date:                 Tue, 24 Feb 2026   Prob (F-statistic):              0.237
Time:                         14:11:34   Log-Likelihood:                -103.64
No. Observations:                   30   AIC:                             211.3
Df Residuals:                       28   BIC:                             214.1
Df Model:                            1
Covariance Type:             nonrobust
==============================================================================
============
                               coef    std err          t      P>|t|      [0.025
0.975]
------------------------------------------------------------------------------
------------
const                        6.7457      1.462      4.613      0.000       3.751
9.741
FFR_AnnualizedChange_bps    -0.0149      0.012     -1.209      0.237      -0.040
0.010
==============================================================================
Omnibus:                       19.978   Durbin-Watson:                   2.116
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               29.185
Skew:                           1.582   Prob(JB):                     4.60e-07
Kurtosis:                       6.651   Cond. No.                         120.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
Intercept: 6.745718627069434, Slope: -0.01486255000436152
```

And then plot the regression line along with the values:

[24]:
```
plot_scatter_regression_ffr_vs_returns(
    cycle_df=treas_10y_cycles,
    asset_label="TREAS_10Y",
    x_vals=X_vals,
    y_vals=Y_vals,
    intercept=model.params[0],
```

```
    slope=model.params[1],
)
```



The above plot is intriguing because of how well the OLS regression appears to fit the data. It certainly appears that during rate-cutting cycles, bonds are an asset that performs well.

0.6.3 High Yield Bonds

Next, we'll run a similar process for high yield bonds using LF98TRUU_Bloomberg US Corporate High Yield Total Return Index Value Unhedged USD.

First, we pull data with the following:

[25]:
```
bb_clean_data(
    base_directory=DATA_DIR,
    fund_ticker_name="LF98TRUU_Bloomberg US Corporate High Yield Total Return␣
 ↪Index Value Unhedged USD",
    source="Bloomberg",
    asset_class="Indices",
    excel_export=True,
    pickle_export=True,
    output_confirmation=False,
)

hy_bonds = load_data(
    base_directory=DATA_DIR,
```

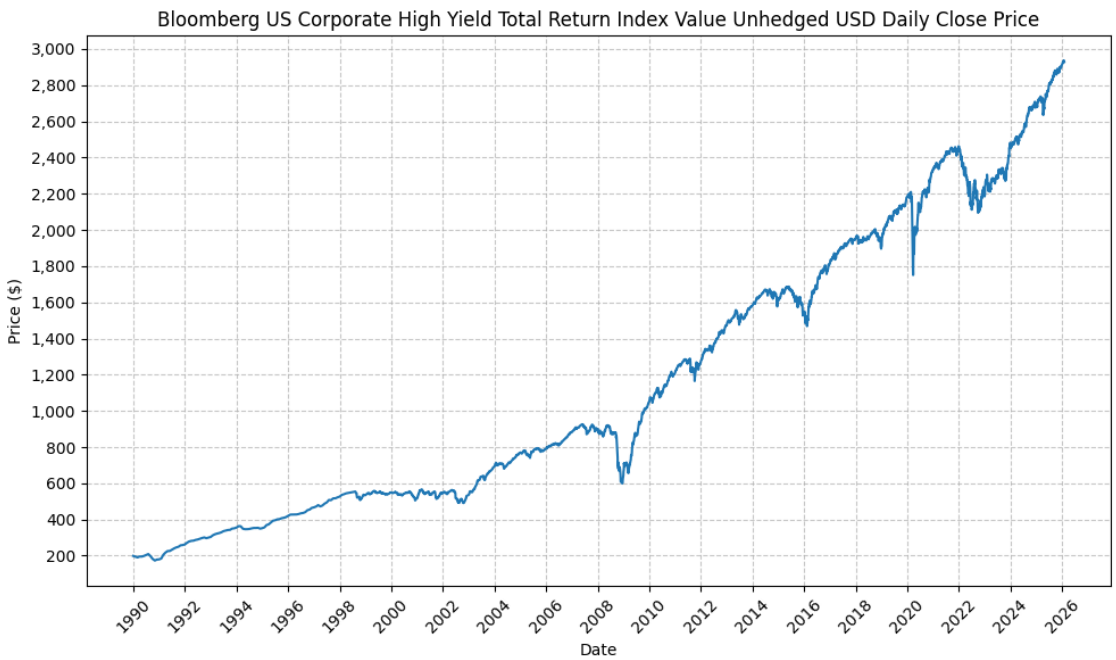
```
    ticker="LF98TRUU_Bloomberg US Corporate High Yield Total Return Index Value␣
 ↪Unhedged USD_Clean",
    source="Bloomberg",
    asset_class="Indices",
    timeframe="Daily",
    file_format="pickle",
)

# Filter HY_BONDS to date range
hy_bonds = hy_bonds[(hy_bonds.index >= pd.to_datetime(start_date)) & (hy_bonds.
 ↪index <= pd.to_datetime(end_date))]

# Drop everything except the "close" column
hy_bonds = hy_bonds[["Close"]]

# Resample to monthly frequency
hy_bonds_monthly = hy_bonds.resample("M").last()
hy_bonds_monthly["Monthly_Return"] = hy_bonds_monthly["Close"].pct_change()
hy_bonds_monthly = hy_bonds_monthly.dropna()

display(hy_bonds_monthly)
```

```
                Close  Monthly_Return
Date
1990-01-31  194.21000        -0.02146
1990-02-28  190.20000        -0.02065
1990-03-31  195.19000         0.02624
1990-04-30  194.86000        -0.00169
1990-05-31  198.62000         0.01930
…                …             …
2025-09-30 2876.85000         0.00816
2025-10-31 2881.38000         0.00157
2025-11-30 2898.07000         0.00579
2025-12-31 2914.49000         0.00567
2026-01-31 2929.32000         0.00509

[433 rows x 2 columns]
```

Next, we can plot the price history before calculating the cycle performance:

[26]:
```
plot_timeseries(
    price_df=hy_bonds,
    plot_start_date=None,
    plot_end_date=None,
    plot_columns=["Close"],
    title="Bloomberg US Corporate High Yield Total Return Index Value Unhedged␣
 ↪USD Daily Close Price",
    x_label="Date",
```

```
    x_format="Year",
    x_tick_spacing=2,
    x_tick_rotation=45,
    y_label="Price ($)",
    y_format="Decimal",
    y_format_decimal_places=0,
    y_tick_spacing=200,
    grid=True,
    legend=False,
    export_plot=False,
    plot_file_name=None,
)
```



Next, we will calculate the performance of high yield bonds based on the pre-defined Fed cycles:

[27]:
```
hy_bonds_cycles = calc_fed_cycle_asset_performance(
    start_date=cycle_ranges["start_date"],
    end_date=cycle_ranges["end_date"],
    label=cycle_ranges["cycle_label"],
    fed_funds_change=cycle_ranges["fed_funds_change"],
    monthly_returns=hy_bonds_monthly,
)

display(hy_bonds_cycles)
```

```
       Cycle        Start          End  Months  CumulativeReturn  \
```

0	Cycle 1	1989-12-31	1990-06-30	6	0.02494
1	Cycle 2	1990-06-30	1993-03-31	34	0.62149
2	Cycle 3	1993-03-31	1994-01-31	11	0.14262
3	Cycle 4	1994-01-31	1995-06-30	18	0.11254
4	Cycle 5	1995-06-30	1996-07-31	14	0.10892
5	Cycle 6	1996-07-31	1997-02-28	8	0.10479
6	Cycle 7	1997-02-28	1997-09-30	8	0.09557
7	Cycle 8	1997-09-30	1998-08-31	12	0.03219
8	Cycle 9	1998-08-31	1999-05-31	10	-0.00728
9	Cycle 10	1999-05-31	2000-11-30	19	-0.08916
10	Cycle 11	2000-11-30	2000-12-31	2	-0.02105
11	Cycle 12	2000-12-31	2002-06-30	19	0.02117
12	Cycle 13	2002-06-30	2002-10-31	5	-0.10874
13	Cycle 14	2002-10-31	2003-12-31	15	0.37662
14	Cycle 15	2003-12-31	2004-05-31	6	0.02188
15	Cycle 16	2004-05-31	2006-12-31	32	0.25625
16	Cycle 17	2006-12-31	2007-08-31	9	0.01683
17	Cycle 18	2007-08-31	2009-07-31	24	0.04908
18	Cycle 19	2009-07-31	2015-11-30	77	0.83146
19	Cycle 20	2015-11-30	2016-06-30	8	0.03948
20	Cycle 21	2016-06-30	2016-11-30	6	0.06426
21	Cycle 22	2016-11-30	2019-06-30	32	0.17306
22	Cycle 23	2019-06-30	2019-07-31	2	0.02856
23	Cycle 24	2019-07-31	2020-09-30	15	0.04631
24	Cycle 25	2020-09-30	2022-02-28	18	0.06775
25	Cycle 26	2022-02-28	2024-01-31	24	0.03580
26	Cycle 27	2024-01-31	2024-08-31	8	0.06285
27	Cycle 28	2024-08-31	2025-06-30	11	0.08182
28	Cycle 29	2025-06-30	2025-08-31	3	0.03577
29	Cycle 30	2025-08-31	2026-01-31	6	0.03937

	CumulativeReturnPct	AverageMonthlyReturn	AverageMonthlyReturnPct	\
0	2.49408	0.00432	0.43159	
1	62.14883	0.01480	1.47959	
2	14.26235	0.01221	1.22129	
3	11.25422	0.00607	0.60701	
4	10.89235	0.00743	0.74262	
5	10.47889	0.01255	1.25548	
6	9.55703	0.01156	1.15581	
7	3.21938	0.00282	0.28169	
8	-0.72813	-0.00042	-0.04217	
9	-8.91594	-0.00480	-0.47963	
10	-2.10510	-0.01014	-1.01430	
11	2.11701	0.00167	0.16693	
12	-10.87431	-0.02215	-2.21511	
13	37.66182	0.02172	2.17230	
14	2.18782	0.00371	0.37115	
15	25.62459	0.00722	0.72211	

16	1.68350	0.00199	0.19914
17	4.90805	0.00371	0.37092
18	83.14599	0.00808	0.80820
19	3.94781	0.00515	0.51508
20	6.42628	0.01049	1.04900
21	17.30568	0.00508	0.50759
22	2.85572	0.01421	1.42144
23	4.63140	0.00372	0.37244
24	6.77496	0.00374	0.37444
25	3.58042	0.00186	0.18605
26	6.28480	0.00769	0.76866
27	8.18158	0.00722	0.72241
28	3.57673	0.01180	1.17991
29	3.93741	0.00646	0.64627

	AnnualizedReturn	AnnualizedReturnPct	Volatility	FedFundsChange	\
0	0.05050	5.05036	0.07625	0.00000	
1	0.18601	18.60069	0.10898	-0.05250	
2	0.15656	15.65571	0.02227	0.00000	
3	0.07369	7.36869	0.05691	0.03000	
4	0.09267	9.26651	0.01906	-0.00750	
5	0.16123	16.12319	0.02366	0.00000	
6	0.14673	14.67279	0.04792	0.00250	
7	0.03219	3.21938	0.06631	0.00000	
8	-0.00873	-0.87312	0.09026	-0.00750	
9	-0.05728	-5.72758	0.05172	0.01750	
10	-0.11984	-11.98427	0.14432	0.00000	
11	0.01332	1.33189	0.11914	-0.04750	
12	-0.24141	-24.14102	0.13364	0.00000	
13	0.29137	29.13703	0.07004	-0.00750	
14	0.04424	4.42350	0.05322	0.00000	
15	0.08931	8.93138	0.04056	0.04250	
16	0.02251	2.25094	0.05992	0.00000	
17	0.02425	2.42463	0.20404	-0.05000	
18	0.09889	9.88931	0.06867	0.00000	
19	0.05980	5.97977	0.09107	0.00250	
20	0.13266	13.26554	0.04025	0.00000	
21	0.06168	6.16825	0.04347	0.02000	
22	0.18405	18.40520	0.04203	0.00000	
23	0.03688	3.68827	0.13113	-0.02250	
24	0.04467	4.46712	0.04952	0.00000	
25	0.01774	1.77447	0.09916	0.05250	
26	0.09574	9.57381	0.03255	0.00000	
27	0.08958	8.95776	0.03617	-0.01000	
28	0.15093	15.09297	0.02414	0.00000	
29	0.08030	8.02985	0.01260	-0.00750	

FedFundsChange_bps FFR_AnnualizedChange FFR_AnnualizedChange_bps \

0	0.00000	0.00000	0.00000
1	-525.00000	-0.01853	-185.29412
2	0.00000	0.00000	0.00000
3	300.00000	0.02000	200.00000
4	-75.00000	-0.00643	-64.28571
5	0.00000	0.00000	0.00000
6	25.00000	0.00375	37.50000
7	0.00000	0.00000	0.00000
8	-75.00000	-0.00900	-90.00000
9	175.00000	0.01105	110.52632
10	0.00000	0.00000	0.00000
11	-475.00000	-0.03000	-300.00000
12	0.00000	0.00000	0.00000
13	-75.00000	-0.00600	-60.00000
14	0.00000	0.00000	0.00000
15	425.00000	0.01594	159.37500
16	0.00000	0.00000	0.00000
17	-500.00000	-0.02500	-250.00000
18	0.00000	0.00000	0.00000
19	25.00000	0.00375	37.50000
20	0.00000	0.00000	0.00000
21	200.00000	0.00750	75.00000
22	0.00000	0.00000	0.00000
23	-225.00000	-0.01800	-180.00000
24	0.00000	0.00000	0.00000
25	525.00000	0.02625	262.50000
26	0.00000	0.00000	0.00000
27	-100.00000	-0.01091	-109.09091
28	0.00000	0.00000	0.00000
29	-75.00000	-0.01500	-150.00000

	Label
0	Cycle 1, 1989-12-31 to 1990-06-30
1	Cycle 2, 1990-06-30 to 1993-03-31
2	Cycle 3, 1993-03-31 to 1994-01-31
3	Cycle 4, 1994-01-31 to 1995-06-30
4	Cycle 5, 1995-06-30 to 1996-07-31
5	Cycle 6, 1996-07-31 to 1997-02-28
6	Cycle 7, 1997-02-28 to 1997-09-30
7	Cycle 8, 1997-09-30 to 1998-08-31
8	Cycle 9, 1998-08-31 to 1999-05-31
9	Cycle 10, 1999-05-31 to 2000-11-30
10	Cycle 11, 2000-11-30 to 2000-12-31
11	Cycle 12, 2000-12-31 to 2002-06-30
12	Cycle 13, 2002-06-30 to 2002-10-31
13	Cycle 14, 2002-10-31 to 2003-12-31
14	Cycle 15, 2003-12-31 to 2004-05-31
15	Cycle 16, 2004-05-31 to 2006-12-31

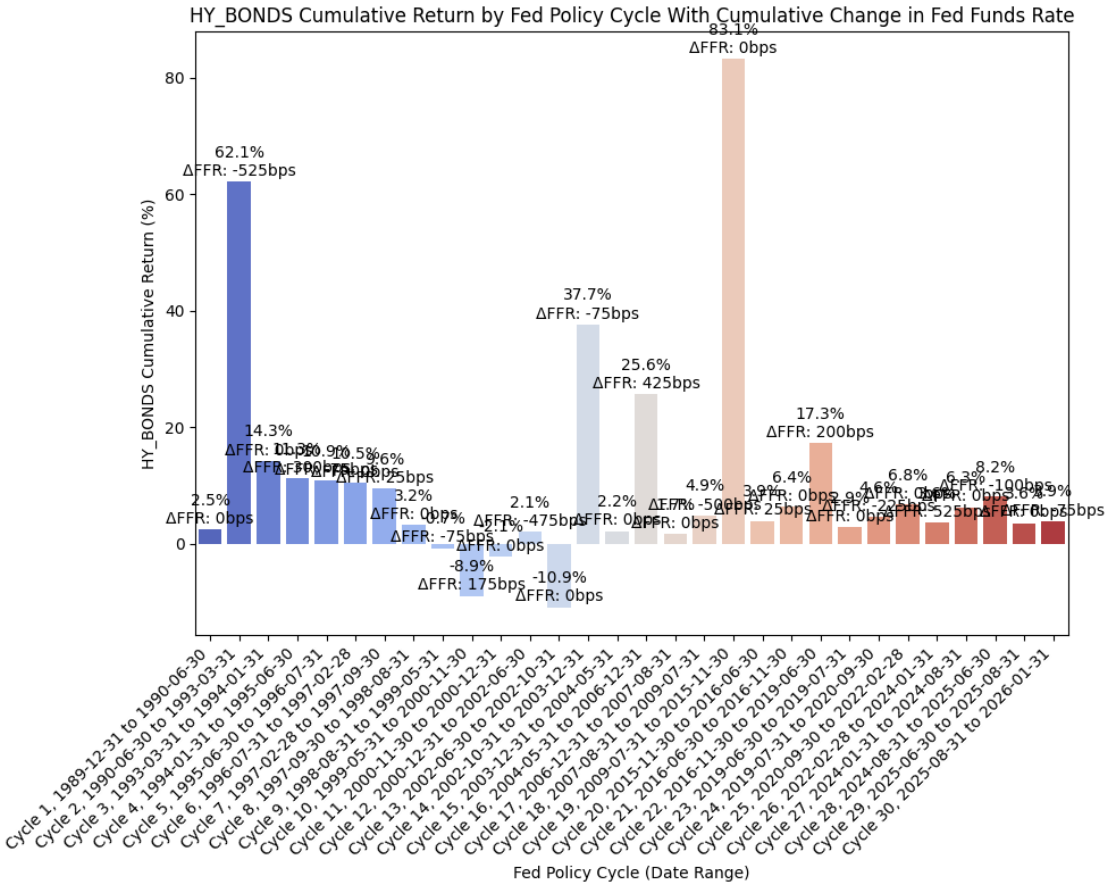
```
16  Cycle 17, 2006-12-31 to 2007-08-31
17  Cycle 18, 2007-08-31 to 2009-07-31
18  Cycle 19, 2009-07-31 to 2015-11-30
19  Cycle 20, 2015-11-30 to 2016-06-30
20  Cycle 21, 2016-06-30 to 2016-11-30
21  Cycle 22, 2016-11-30 to 2019-06-30
22  Cycle 23, 2019-06-30 to 2019-07-31
23  Cycle 24, 2019-07-31 to 2020-09-30
24  Cycle 25, 2020-09-30 to 2022-02-28
25  Cycle 26, 2022-02-28 to 2024-01-31
26  Cycle 27, 2024-01-31 to 2024-08-31
27  Cycle 28, 2024-08-31 to 2025-06-30
28  Cycle 29, 2025-06-30 to 2025-08-31
29  Cycle 30, 2025-08-31 to 2026-01-31
```

This gives us the following data points:

- Cycle start date
- Cycle end date
- Number of months in the cycle
- Cumulative return during the cycle (decimal and percent)
- Average monthly return during the cycle (decimal and percent)
- Annualized return during the cycle (decimal and percent)
- Return volatility during the cycle
- Cumulative change in FFR during the cycle (decimal and basis points)
- Annualized change in FFR during the cycle (decimal and basis points)

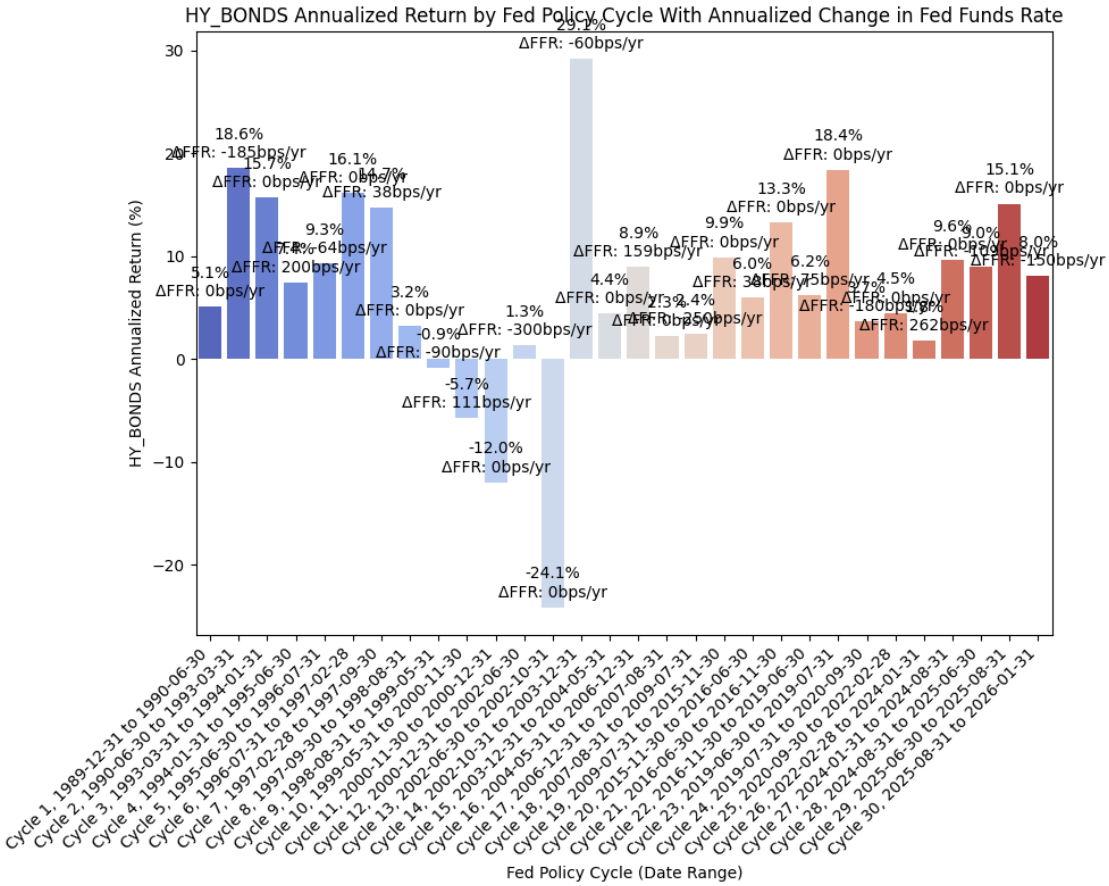
From the above DataFrame, we can then plot the cumulative and annualized returns for each cycle in a bar chart. First, the cumulative returns along with the cumulative change in FFR:

[28]:
```
plot_bar_returns_ffr_change(
    cycle_df=hy_bonds_cycles,
    asset_label="HY_BONDS",
    annualized_or_cumulative="Cumulative",
)
```



And then the annualized returns along with the annualized change in FFR:

[29]:
```
plot_bar_returns_ffr_change(
    cycle_df=hy_bonds_cycles,
    asset_label="HY_BONDS",
    annualized_or_cumulative="Annualized",
)
```



Finally, we can run an OLS regression to check fit:

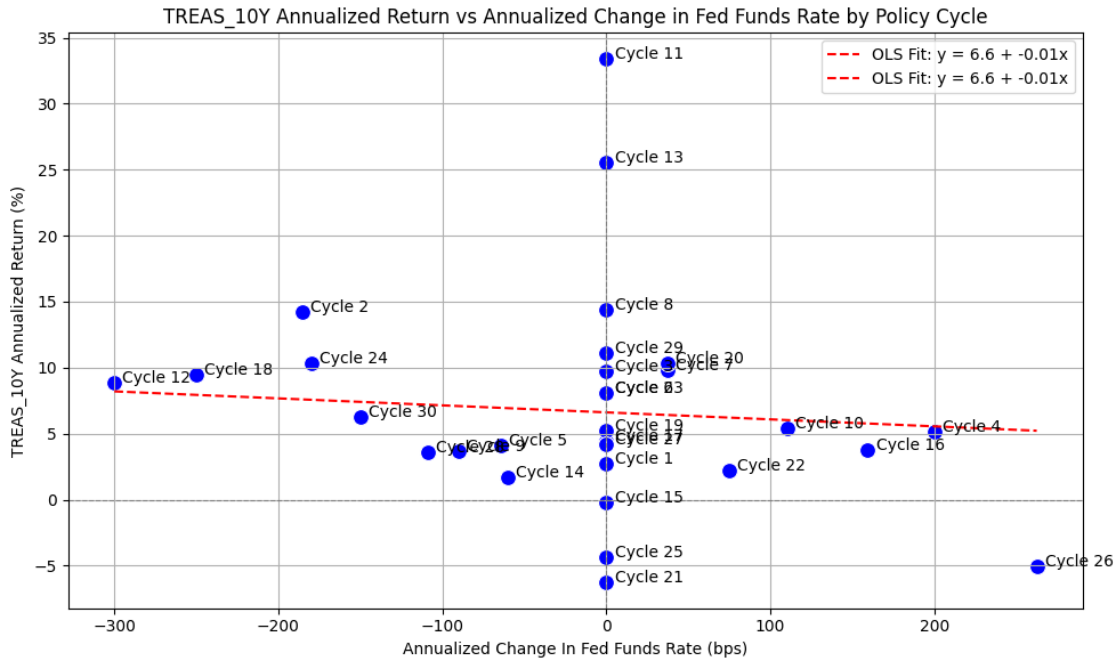
[30]:
```
df = hy_bonds_cycles

#=== Don't modify below this line ===

# Run OLS regression with statsmodels
X = df["FFR_AnnualizedChange_bps"]
y = df["AnnualizedReturnPct"]
X = sm.add_constant(X)
model = sm.OLS(y, X).fit()
print(model.summary())
print(f"Intercept: {model.params[0]}, Slope: {model.params[1]}")  # Intercept␣
↪and slope

# Calc X and Y values for regression line
X_vals = np.linspace(X.min(), X.max(), 100)
Y_vals = model.params[0] + model.params[1] * X_vals
```

```
                            OLS Regression Results
```

```
==============================================================================
Dep. Variable:     AnnualizedReturnPct   R-squared:                       0.004
Model:                             OLS   Adj. R-squared:                 -0.031
Method:                  Least Squares   F-statistic:                    0.1171
Date:                 Tue, 24 Feb 2026   Prob (F-statistic):              0.735
Time:                         14:11:36   Log-Likelihood:                -110.57
No. Observations:                   30   AIC:                             225.1
Df Residuals:                       28   BIC:                             227.9
Df Model:                            1
Covariance Type:             nonrobust
==============================================================================================
                                coef    std err          t      P>|t|      [0.025      0.975]
----------------------------------------------------------------------------------------------
const                         6.6113      1.842      3.590      0.001       2.839      10.384
FFR_AnnualizedChange_bps     -0.0053      0.015     -0.342      0.735      -0.037       0.026
==============================================================================
Omnibus:                        9.087   Durbin-Watson:                   1.936
Prob(Omnibus):                  0.011   Jarque-Bera (JB):                9.047
Skew:                          -0.790   Prob(JB):                       0.0109
Kurtosis:                       5.178   Cond. No.                         120.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
Intercept: 6.611340605920211, Slope: -0.005299161729553994
```

And then plot the regression line along with the values:

[31]:
```
plot_scatter_regression_ffr_vs_returns(
    cycle_df=treas_10y_cycles,
    asset_label="TREAS_10Y",
    x_vals=X_vals,
    y_vals=Y_vals,
    intercept=model.params[0],
    slope=model.params[1],
)
```



0.6.4 Gold

Finally, we'll run a similar process for gold using XAU_Gold USD Spot.

First, we pull data with the following:

[32]:
```
bb_clean_data(
    base_directory=DATA_DIR,
    fund_ticker_name="XAU_Gold USD Spot",
    source="Bloomberg",
    asset_class="Commodities",
    excel_export=True,
    pickle_export=True,
    output_confirmation=False,
)

gold = load_data(
    base_directory=DATA_DIR,
    ticker="XAU_Gold USD Spot_Clean",
    source="Bloomberg",
    asset_class="Commodities",
    timeframe="Daily",
    file_format="pickle",
)

# Filter GOLD to date range
```

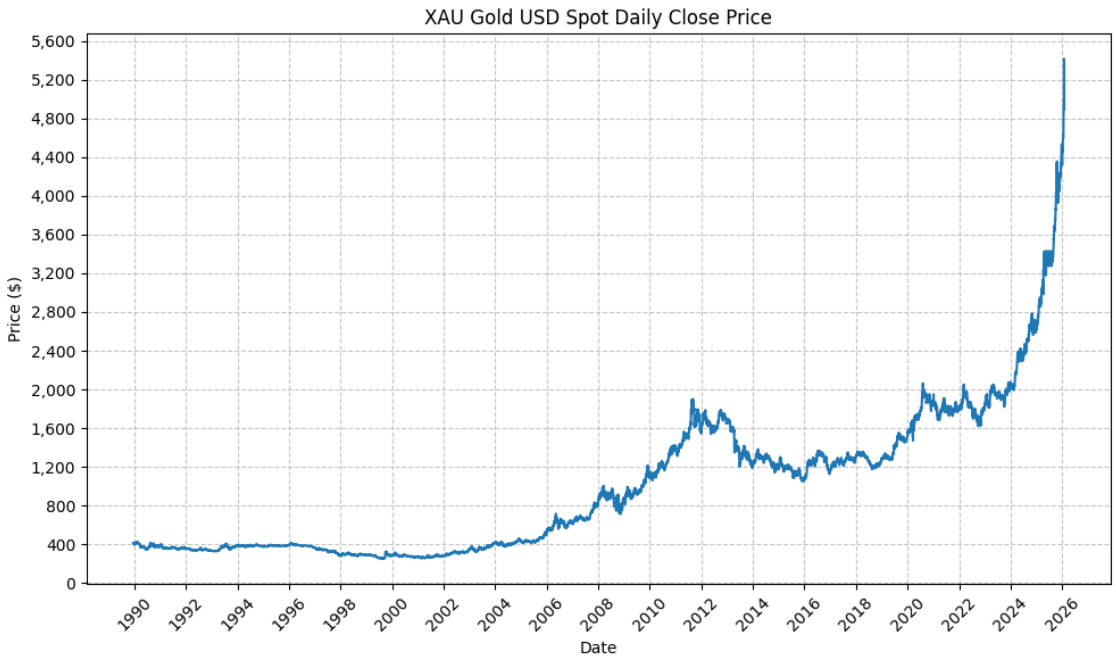
```
gold = gold[(gold.index >= pd.to_datetime(start_date)) & (gold.index <= pd.
 ↪to_datetime(end_date))]

# Drop everything except the "close" column
gold = gold[["Close"]]

# Resample to monthly frequency
gold_monthly = gold.resample("M").last()
gold_monthly["Monthly_Return"] = gold_monthly["Close"].pct_change()
gold_monthly = gold_monthly.dropna()

display(gold_monthly)
```

```
                Close  Monthly_Return
Date
1990-01-31  415.05000         0.03439
1990-02-28  407.70000        -0.01771
1990-03-31  368.50000        -0.09615
1990-04-30  367.75000        -0.00204
1990-05-31  363.05000        -0.01278
…                …             …
2025-09-30 3858.96000         0.11920
2025-10-31 4002.92000         0.03731
2025-11-30 4239.43000         0.05908
2025-12-31 4319.37000         0.01886
2026-01-31 4894.23000         0.13309

[433 rows x 2 columns]
```

Next, we can plot the price history before calculating the cycle performance:

[33]:
```
plot_timeseries(
    price_df=gold,
    plot_start_date=None,
    plot_end_date=None,
    plot_columns=["Close"],
    title="XAU Gold USD Spot Daily Close Price",
    x_label="Date",
    x_format="Year",
    x_tick_spacing=2,
    x_tick_rotation=45,
    y_label="Price ($)",
    y_format="Decimal",
    y_format_decimal_places=0,
    y_tick_spacing=400,
    grid=True,
    legend=False,
    export_plot=False,
```

```
    plot_file_name=None,
)
```



Next, we will calculate the performance of gold based on the pre-defined Fed cycles:

[34]:
```
gold_cycles = calc_fed_cycle_asset_performance(
    start_date=cycle_ranges["start_date"],
    end_date=cycle_ranges["end_date"],
    label=cycle_ranges["cycle_label"],
    fed_funds_change=cycle_ranges["fed_funds_change"],
    monthly_returns=gold_monthly,
)

display(gold_cycles)
```

```
       Cycle       Start         End  Months  CumulativeReturn  \
0    Cycle 1  1989-12-31  1990-06-30       6          -0.12224
1    Cycle 2  1990-06-30  1993-03-31      34          -0.06624
2    Cycle 3  1993-03-31  1994-01-31      11           0.15903
3    Cycle 4  1994-01-31  1995-06-30      18          -0.01561
4    Cycle 5  1995-06-30  1996-07-31      14           0.00716
5    Cycle 6  1996-07-31  1997-02-28       8          -0.04468
6    Cycle 7  1997-02-28  1997-09-30       8          -0.02875
7    Cycle 8  1997-09-30  1998-08-31      12          -0.14993
8    Cycle 9  1998-08-31  1999-05-31      10          -0.05621
9   Cycle 10  1999-05-31  2000-11-30      19          -0.05619
```

10	Cycle 11	2000-11-30	2000-12-31	2	0.02678
11	Cycle 12	2000-12-31	2002-06-30	19	0.16269
12	Cycle 13	2002-06-30	2002-10-31	5	-0.02695
13	Cycle 14	2002-10-31	2003-12-31	15	0.28404
14	Cycle 15	2003-12-31	2004-05-31	6	-0.00653
15	Cycle 16	2004-05-31	2006-12-31	32	0.64628
16	Cycle 17	2006-12-31	2007-08-31	9	0.03904
17	Cycle 18	2007-08-31	2009-07-31	24	0.43610
18	Cycle 19	2009-07-31	2015-11-30	77	0.14924
19	Cycle 20	2015-11-30	2016-06-30	8	0.15742
20	Cycle 21	2016-06-30	2016-11-30	6	-0.03466
21	Cycle 22	2016-11-30	2019-06-30	32	0.10362
22	Cycle 23	2019-06-30	2019-07-31	2	0.08288
23	Cycle 24	2019-07-31	2020-09-30	15	0.33789
24	Cycle 25	2020-09-30	2022-02-28	18	-0.02989
25	Cycle 26	2022-02-28	2024-01-31	24	0.13485
26	Cycle 27	2024-01-31	2024-08-31	8	0.21348
27	Cycle 28	2024-08-31	2025-06-30	11	0.34954
28	Cycle 29	2025-06-30	2025-08-31	3	0.04825
29	Cycle 30	2025-08-31	2026-01-31	6	0.48764

	CumulativeReturnPct	AverageMonthlyReturn	AverageMonthlyReturnPct	\
0	-12.22430	-0.02069	-2.06945	
1	-6.62443	-0.00158	-0.15839	
2	15.90288	0.01462	1.46238	
3	-1.56130	-0.00070	-0.06966	
4	0.71559	0.00066	0.06552	
5	-4.46839	-0.00519	-0.51936	
6	-2.87498	-0.00313	-0.31292	
7	-14.99306	-0.01285	-1.28489	
8	-5.62053	-0.00517	-0.51718	
9	-5.61857	-0.00193	-0.19269	
10	2.67773	0.01332	1.33221	
11	16.26918	0.00844	0.84406	
12	-2.69484	-0.00496	-0.49607	
13	28.40365	0.01772	1.77192	
14	-0.65302	0.00044	0.04419	
15	64.62831	0.01653	1.65326	
16	3.90432	0.00447	0.44660	
17	43.60981	0.01766	1.76641	
18	14.92391	0.00317	0.31742	
19	15.74192	0.02027	2.02679	
20	-3.46575	-0.00449	-0.44887	
21	10.36165	0.00355	0.35519	
22	8.28750	0.04132	4.13180	
23	33.78880	0.02043	2.04336	
24	-2.98862	-0.00076	-0.07561	
25	13.48509	0.00605	0.60516	

26	21.34824	0.02494	2.49353
27	34.95424	0.02824	2.82419
28	4.82481	0.01609	1.60850
29	48.76396	0.06926	6.92618

	AnnualizedReturn	AnnualizedReturnPct	Volatility	FedFundsChange	\
0	-0.22954	-22.95426	0.14885	0.00000	
1	-0.02390	-2.39005	0.10292	-0.05250	
2	0.17468	17.46838	0.17075	0.00000	
3	-0.01044	-1.04359	0.06717	0.03000	
4	0.00613	0.61305	0.06186	-0.00750	
5	-0.06627	-6.62715	0.11737	0.00000	
6	-0.04281	-4.28133	0.11886	0.00250	
7	-0.14993	-14.99306	0.12433	0.00000	
8	-0.06706	-6.70614	0.12709	-0.00750	
9	-0.03586	-3.58627	0.17333	0.01750	
10	0.17181	17.18109	0.03266	0.00000	
11	0.09988	9.98819	0.11022	-0.04750	
12	-0.06346	-6.34605	0.12104	0.00000	
13	0.22141	22.14112	0.15439	-0.00750	
14	-0.01302	-1.30178	0.20868	0.00000	
15	0.20556	20.55610	0.14568	0.04250	
16	0.05239	5.23935	0.07374	0.00000	
17	0.19837	19.83731	0.24658	-0.05000	
18	0.02191	2.19146	0.18253	0.00000	
19	0.24519	24.51911	0.22531	0.00250	
20	-0.06811	-6.81139	0.19940	0.00000	
21	0.03766	3.76642	0.10758	0.02000	
22	0.61239	61.23899	0.18771	0.00000	
23	0.26222	26.22222	0.14899	-0.02250	
24	-0.02002	-2.00247	0.15388	0.00000	
25	0.06529	6.52938	0.13996	0.05250	
26	0.33675	33.67502	0.11399	0.00000	
27	0.38683	38.68250	0.12923	-0.01000	
28	0.20741	20.74143	0.09689	0.00000	
29	1.21307	121.30714	0.16013	-0.00750	

	FedFundsChange_bps	FFR_AnnualizedChange	FFR_AnnualizedChange_bps	\
0	0.00000	0.00000	0.00000	
1	-525.00000	-0.01853	-185.29412	
2	0.00000	0.00000	0.00000	
3	300.00000	0.02000	200.00000	
4	-75.00000	-0.00643	-64.28571	
5	0.00000	0.00000	0.00000	
6	25.00000	0.00375	37.50000	
7	0.00000	0.00000	0.00000	
8	-75.00000	-0.00900	-90.00000	
9	175.00000	0.01105	110.52632	

10	0.00000	0.00000	0.00000
11	-475.00000	-0.03000	-300.00000
12	0.00000	0.00000	0.00000
13	-75.00000	-0.00600	-60.00000
14	0.00000	0.00000	0.00000
15	425.00000	0.01594	159.37500
16	0.00000	0.00000	0.00000
17	-500.00000	-0.02500	-250.00000
18	0.00000	0.00000	0.00000
19	25.00000	0.00375	37.50000
20	0.00000	0.00000	0.00000
21	200.00000	0.00750	75.00000
22	0.00000	0.00000	0.00000
23	-225.00000	-0.01800	-180.00000
24	0.00000	0.00000	0.00000
25	525.00000	0.02625	262.50000
26	0.00000	0.00000	0.00000
27	-100.00000	-0.01091	-109.09091
28	0.00000	0.00000	0.00000
29	-75.00000	-0.01500	-150.00000

	Label
0	Cycle 1, 1989-12-31 to 1990-06-30
1	Cycle 2, 1990-06-30 to 1993-03-31
2	Cycle 3, 1993-03-31 to 1994-01-31
3	Cycle 4, 1994-01-31 to 1995-06-30
4	Cycle 5, 1995-06-30 to 1996-07-31
5	Cycle 6, 1996-07-31 to 1997-02-28
6	Cycle 7, 1997-02-28 to 1997-09-30
7	Cycle 8, 1997-09-30 to 1998-08-31
8	Cycle 9, 1998-08-31 to 1999-05-31
9	Cycle 10, 1999-05-31 to 2000-11-30
10	Cycle 11, 2000-11-30 to 2000-12-31
11	Cycle 12, 2000-12-31 to 2002-06-30
12	Cycle 13, 2002-06-30 to 2002-10-31
13	Cycle 14, 2002-10-31 to 2003-12-31
14	Cycle 15, 2003-12-31 to 2004-05-31
15	Cycle 16, 2004-05-31 to 2006-12-31
16	Cycle 17, 2006-12-31 to 2007-08-31
17	Cycle 18, 2007-08-31 to 2009-07-31
18	Cycle 19, 2009-07-31 to 2015-11-30
19	Cycle 20, 2015-11-30 to 2016-06-30
20	Cycle 21, 2016-06-30 to 2016-11-30
21	Cycle 22, 2016-11-30 to 2019-06-30
22	Cycle 23, 2019-06-30 to 2019-07-31
23	Cycle 24, 2019-07-31 to 2020-09-30
24	Cycle 25, 2020-09-30 to 2022-02-28
25	Cycle 26, 2022-02-28 to 2024-01-31

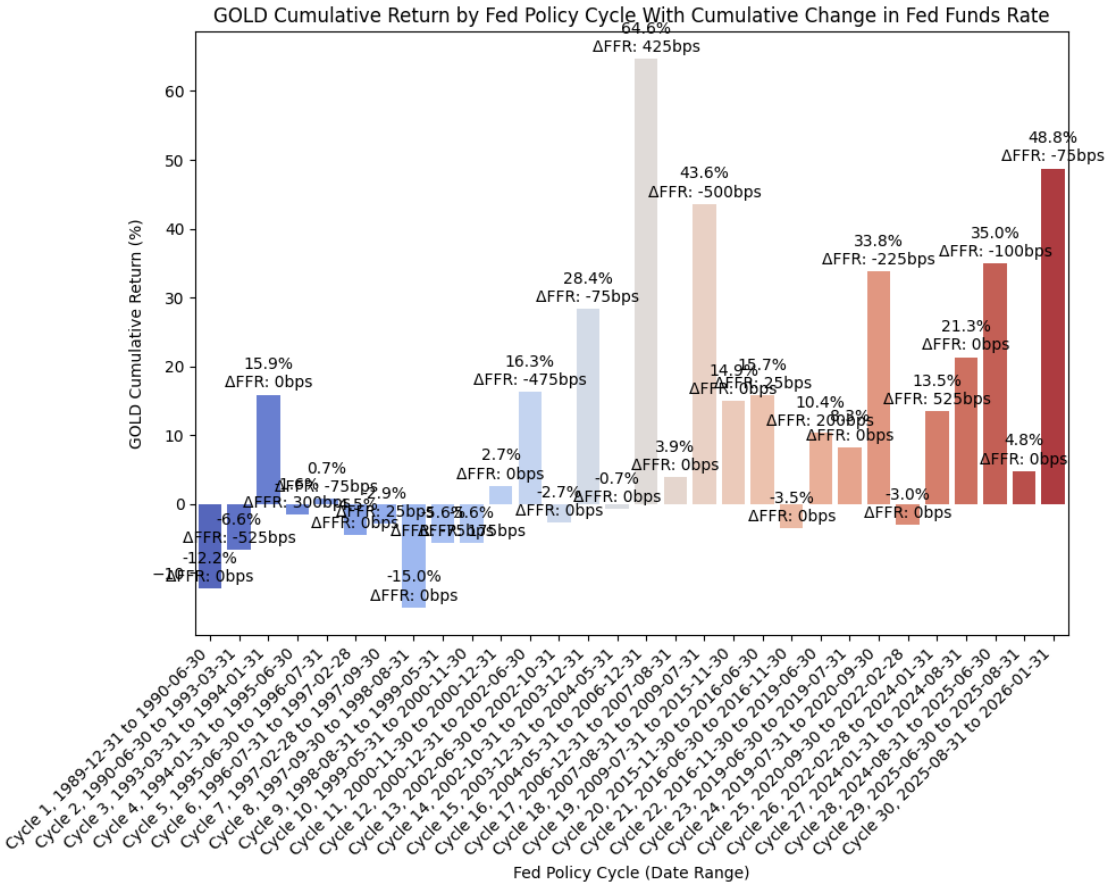
```
26  Cycle 27, 2024-01-31 to 2024-08-31
27  Cycle 28, 2024-08-31 to 2025-06-30
28  Cycle 29, 2025-06-30 to 2025-08-31
29  Cycle 30, 2025-08-31 to 2026-01-31
```

This gives us the following data points:

- Cycle start date
- Cycle end date
- Number of months in the cycle
- Cumulative return during the cycle (decimal and percent)
- Average monthly return during the cycle (decimal and percent)
- Annualized return during the cycle (decimal and percent)
- Return volatility during the cycle
- Cumulative change in FFR during the cycle (decimal and basis points)
- Annualized change in FFR during the cycle (decimal and basis points)

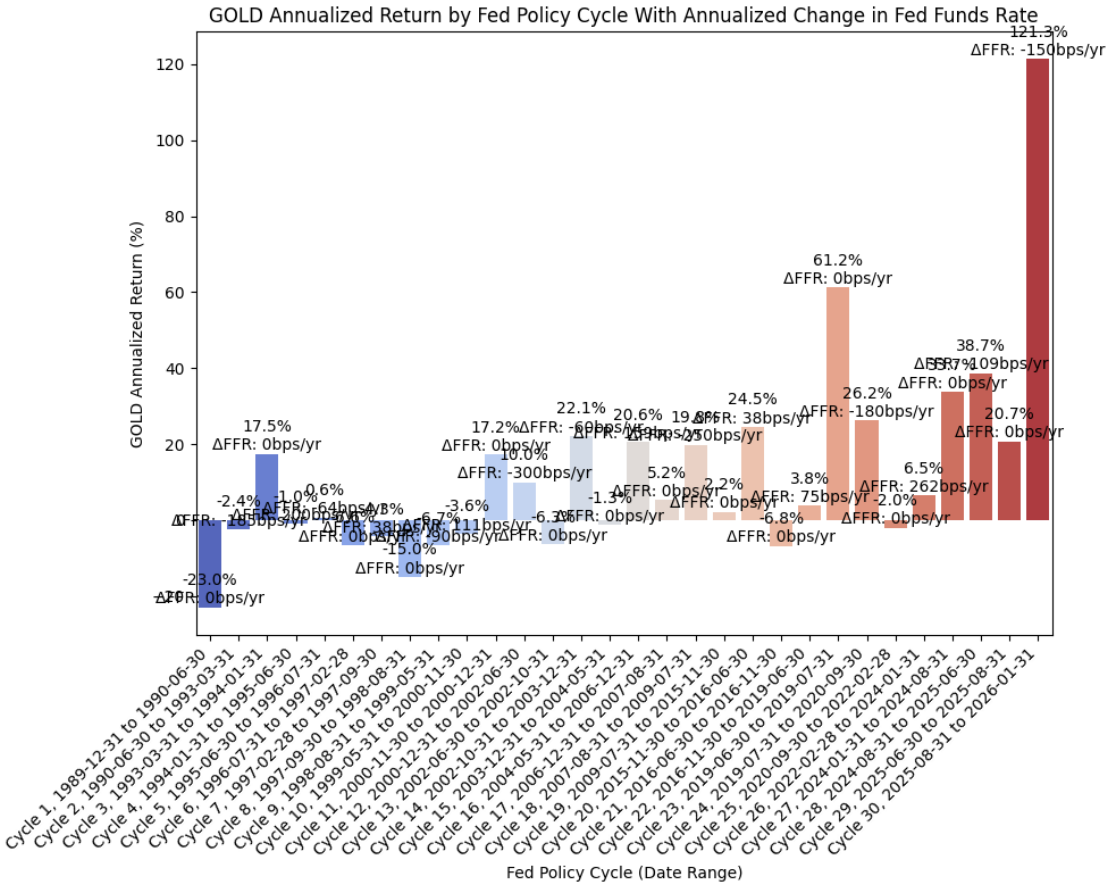
From the above DataFrame, we can then plot the cumulative and annualized returns for each cycle in a bar chart. First, the cumulative returns along with the cumulative change in FFR:

[35]:
```
plot_bar_returns_ffr_change(
    cycle_df=gold_cycles,
    asset_label="GOLD",
    annualized_or_cumulative="Cumulative",
)
```



And then the annualized returns along with the annualized change in FFR:

[36]:
```
plot_bar_returns_ffr_change(
    cycle_df=gold_cycles,
    asset_label="GOLD",
    annualized_or_cumulative="Annualized",
)
```



We see strong returns for gold across several different Fed cycles, so it is difficult to draw any kind of initial conclusion based on the bar charts.

Finally, we can run an OLS regression to check fit:

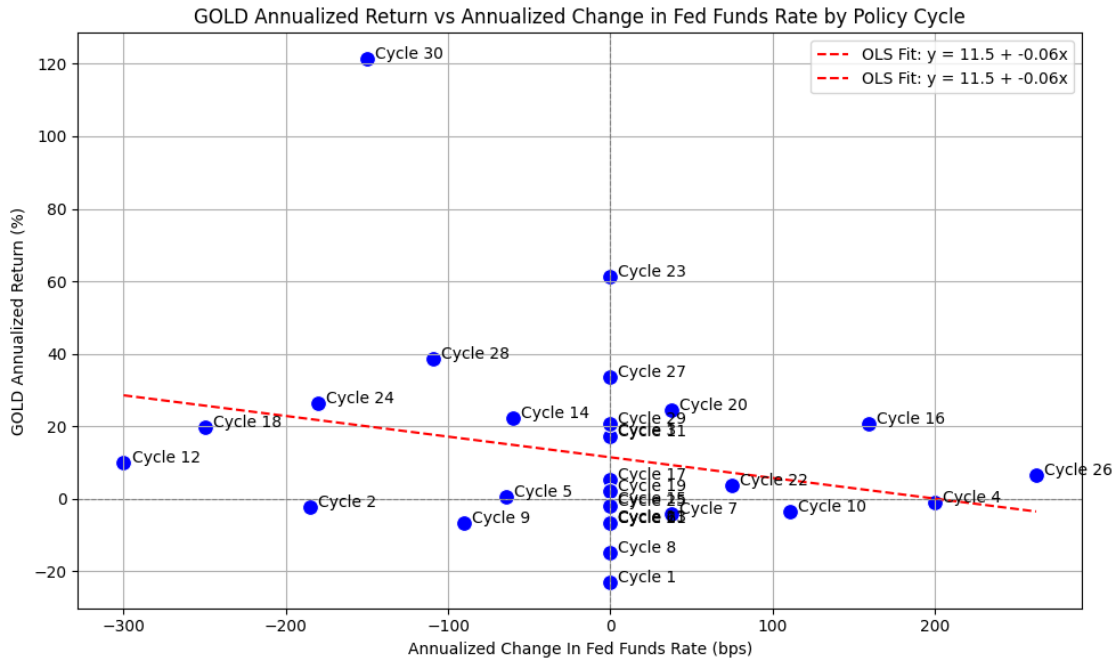
[37]:
```
df = gold_cycles

#=== Don't modify below this line ===

# Run OLS regression with statsmodels
X = df["FFR_AnnualizedChange_bps"]
y = df["AnnualizedReturnPct"]
X = sm.add_constant(X)
model = sm.OLS(y, X).fit()
print(model.summary())
print(f"Intercept: {model.params[0]}, Slope: {model.params[1]}")  # Intercept␣
↪and slope

# Calc X and Y values for regression line
X_vals = np.linspace(X.min(), X.max(), 100)
```

```
Y_vals = model.params[0] + model.params[1] * X_vals
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:     AnnualizedReturnPct   R-squared:                       0.064
Model:                             OLS   Adj. R-squared:                  0.030
Method:                  Least Squares   F-statistic:                     1.900
Date:                 Tue, 24 Feb 2026   Prob (F-statistic):              0.179
Time:                         14:11:39   Log-Likelihood:                -140.03
No. Observations:                   30   AIC:                             284.1
Df Residuals:                       28   BIC:                             286.9
Df Model:                            1
Covariance Type:             nonrobust
==================================================================================
============
                                coef    std err          t      P>|t|      [0.025
0.975]
----------------------------------------------------------------------------------
------------
const                        11.4670      4.917      2.332      0.027       1.396
21.539
FFR_AnnualizedChange_bps     -0.0570      0.041     -1.378      0.179      -0.142
0.028
==============================================================================
Omnibus:                       28.991   Durbin-Watson:                   1.081
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               62.894
Skew:                           2.093   Prob(JB):                     2.20e-14
Kurtosis:                       8.727   Cond. No.                         120.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
Intercept: 11.467013480569243, Slope: -0.056974234027166226
```

And then plot the regression line along with the values:

[38]:
```
plot_scatter_regression_ffr_vs_returns(
    cycle_df=gold_cycles,
    asset_label="GOLD",
    x_vals=X_vals,
    y_vals=Y_vals,
    intercept=model.params[0],
    slope=model.params[1],
)
```



It's difficult to draw any strong conclusions with the above plot. Gold has traditionally been considered a hedge for inflation, and while one of the Fed's mandates is to manage inflation, there may not be a conclusion to draw in relationship to the historical returns that gold has exhibited.

0.7 Asset Allocation

With the above analysis (somewhat) complete, let's look at the optimal allocation for a portfolio based on the data and the hypythetical historical results.

We have to be careful with our criteria for when to hold stocks, bonds, or gold, as hindsight bias is certainly possible. So, without overanalyzing the results, let's assume that we hold stocks as the default position during tightening cycles, and then hold bonds during easing cycles when the Fed starts cutting rates, and then resume holding stocks when the Fed stops cutting rates. If there is not any change in FFR, then we still hold stocks.

We can then combine the return series based on the above with the following:

[39]:
```
# Shift the "cycle_filled" column down by one row to create a new column called␣
↪"cycle_invested" that represents the cycle label that an investor would be␣
↪invested in for each month (i.e. the cycle label from the previous month)

fedfunds_grouped_cycles['cycle_invested'] =␣
↪fedfunds_grouped_cycles['cycle_filled'].shift(1)

display(fedfunds_grouped_cycles)
```

```
    start_date  fed_funds_start   end_date  fed_funds_end  fed_funds_change  \
0   1989-12-31          0.08250 1990-01-31        0.08250           0.00000
```

```
1   1990-01-31          0.08250 1990-02-28        0.08250           0.00000
2   1990-02-28          0.08250 1990-03-31        0.08250           0.00000
3   1990-03-31          0.08250 1990-04-30        0.08250           0.00000
4   1990-04-30          0.08250 1990-05-31        0.08250           0.00000
..         …                  …        …             …                 …
429 2025-09-30          0.04250 2025-10-31        0.04000          -0.00250
430 2025-10-31          0.04000 2025-11-30        0.04000           0.00000
431 2025-11-30          0.04000 2025-12-31        0.03750          -0.00250
432 2025-12-31          0.03750 2026-01-31        0.03750           0.00000
433 2026-01-31          0.03750        NaT            NaN               NaN

       cycle         cycle_filled  group       cycle_invested
0    Neutral  Modified Tightening      1                 None
1    Neutral  Modified Tightening      1  Modified Tightening
2    Neutral  Modified Tightening      1  Modified Tightening
3    Neutral  Modified Tightening      1  Modified Tightening
4    Neutral  Modified Tightening      1  Modified Tightening
..       …                   …    …                   …
429   Easing                Easing     30               Easing
430  Neutral                Easing     30               Easing
431   Easing                Easing     30               Easing
432  Neutral                Easing     30               Easing
433  Neutral                Easing     30               Easing

[434 rows x 9 columns]
```

[40]:
```
# Reset index to merge on date
stocks_merged = pd.merge_asof(
    spxt_monthly.reset_index(),
    fedfunds_grouped_cycles[['start_date', 'end_date', 'cycle_invested',␣
 ↪'group']],
    left_on='Date',
    right_on='start_date',
    direction='backward'
)

# Drop rows where the date falls outside the cycle's end_date
stocks_merged = stocks_merged[stocks_merged['Date'] <=␣
 ↪stocks_merged['end_date']]

display(stocks_merged)
```

```
          Date       Close  Monthly_Return start_date   end_date  \
0   1990-01-31   353.94000        -0.06713 1990-01-31 1990-02-28
1   1990-02-28   358.50000         0.01288 1990-02-28 1990-03-31
2   1990-03-31         368         0.02650 1990-03-31 1990-04-30
3   1990-04-30   358.81000        -0.02497 1990-04-30 1990-05-31
4   1990-05-31   393.80000         0.09752 1990-05-31 1990-06-30
```

```
..          …          …                …         …         …
427 2025-08-31 14304.68000          0.02027 2025-08-31 2025-09-30
428 2025-09-30 14826.80000          0.03650 2025-09-30 2025-10-31
429 2025-10-31 15173.95000          0.02341 2025-10-31 2025-11-30
430 2025-11-30 15211.14000          0.00245 2025-11-30 2025-12-31
431 2025-12-31 15220.45000          0.00061 2025-12-31 2026-01-31

          cycle_invested  group
0    Modified Tightening      1
1    Modified Tightening      1
2    Modified Tightening      1
3    Modified Tightening      1
4    Modified Tightening      1
..                    …    …
427  Modified Tightening     30
428               Easing     30
429               Easing     30
430               Easing     30
431               Easing     30

[432 rows x 7 columns]
```

[41]:
```python
# Reset index to merge on date
bonds_merged = pd.merge_asof(
    treas_10y_monthly.reset_index(),
    fedfunds_grouped_cycles[['start_date', 'end_date', 'cycle_invested',␣
 ↪'group']],
    left_on='Date',
    right_on='start_date',
    direction='backward'
)

# Drop rows where the date falls outside the cycle's end_date
bonds_merged = bonds_merged[bonds_merged['Date'] <= bonds_merged['end_date']]

display(bonds_merged)
```

```
          Date     Close  Monthly_Return start_date   end_date  \
0   1990-01-31  98.01300        -0.01987 1990-01-31 1990-02-28
1   1990-02-28  97.99000        -0.00023 1990-02-28 1990-03-31
2   1990-03-31  97.98900        -0.00001 1990-03-31 1990-04-30
3   1990-04-30  96.60600        -0.01411 1990-04-30 1990-05-31
4   1990-05-31  99.64700         0.03148 1990-05-31 1990-06-30
..          …       …              …         …         …
427 2025-08-31 641.25800         0.01668 2025-08-31 2025-09-30
428 2025-09-30 645.58400         0.00675 2025-09-30 2025-10-31
429 2025-10-31 650.00500         0.00685 2025-10-31 2025-11-30
430 2025-11-30 656.63600         0.01020 2025-11-30 2025-12-31
```

```
431 2025-12-31 652.28800        -0.00662 2025-12-31 2026-01-31

          cycle_invested  group
0    Modified Tightening      1
1    Modified Tightening      1
2    Modified Tightening      1
3    Modified Tightening      1
4    Modified Tightening      1
..                     …    …
427  Modified Tightening     30
428               Easing     30
429               Easing     30
430               Easing     30
431               Easing     30

[432 rows x 7 columns]
```

[42]:
```python
# Select the appropriate return based on cycle
stocks_merged['strategy_return'] = stocks_merged.apply(
    lambda row: row['Monthly_Return'] if row['cycle_invested'] in␣
 ↪['Tightening', 'Modified Tightening'] else None,
    axis=1
)

bonds_merged['strategy_return'] = bonds_merged.apply(
    lambda row: row['Monthly_Return'] if row['cycle_invested'] == 'Easing' else␣
 ↪None,
    axis=1
)

# Combine
strategy = pd.concat([stocks_merged, bonds_merged]).
 ↪dropna(subset=['strategy_return'])
strategy = strategy.sort_values('Date')

display(strategy.head(20))
```

```
         Date     Close  Monthly_Return start_date   end_date  \
0  1990-01-31 353.94000        -0.06713 1990-01-31 1990-02-28
1  1990-02-28 358.50000         0.01288 1990-02-28 1990-03-31
2  1990-03-31       368         0.02650 1990-03-31 1990-04-30
3  1990-04-30 358.81000        -0.02497 1990-04-30 1990-05-31
4  1990-05-31 393.80000         0.09752 1990-05-31 1990-06-30
5  1990-06-30 391.14000        -0.00675 1990-06-30 1990-07-31
6  1990-07-31 102.84500         0.01463 1990-07-31 1990-08-31
7  1990-08-31 100.78200        -0.02006 1990-08-31 1990-09-30
8  1990-09-30 101.74000         0.00951 1990-09-30 1990-10-31
9  1990-10-31 103.65200         0.01879 1990-10-31 1990-11-30
```

```
10 1990-11-30 106.43100         0.02681 1990-11-30 1990-12-31
11 1990-12-31 108.24900         0.01708 1990-12-31 1991-01-31
12 1991-01-31 109.52500         0.01179 1991-01-31 1991-02-28
13 1991-02-28 110.10400         0.00529 1991-02-28 1991-03-31
14 1991-03-31 110.43400         0.00300 1991-03-31 1991-04-30
15 1991-04-30 111.59800         0.01054 1991-04-30 1991-05-31
16 1991-05-31 112.07800         0.00430 1991-05-31 1991-06-30
17 1991-06-30 111.50200        -0.00514 1991-06-30 1991-07-31
18 1991-07-31 113.05200         0.01390 1991-07-31 1991-08-31
19 1991-08-31 116.06500         0.02665 1991-08-31 1991-09-30

         cycle_invested  group  strategy_return
0   Modified Tightening      1         -0.06713
1   Modified Tightening      1          0.01288
2   Modified Tightening      1          0.02650
3   Modified Tightening      1         -0.02497
4   Modified Tightening      1          0.09752
5   Modified Tightening      2         -0.00675
6               Easing      2          0.01463
7               Easing      2         -0.02006
8               Easing      2          0.00951
9               Easing      2          0.01879
10              Easing      2          0.02681
11              Easing      2          0.01708
12              Easing      2          0.01179
13              Easing      2          0.00529
14              Easing      2          0.00300
15              Easing      2          0.01054
16              Easing      2          0.00430
17              Easing      2         -0.00514
18              Easing      2          0.01390
19              Easing      2          0.02665
```

[43]:
```
# Calculate cumulative returns and drawdown for spxt
spxt_monthly['Cumulative_Return'] = (1 + spxt_monthly['Monthly_Return']).
 ↪cumprod() - 1
spxt_monthly['Cumulative_Return_Plus_One'] = 1 +␣
 ↪spxt_monthly['Cumulative_Return']
spxt_monthly['Rolling_Max'] = spxt_monthly['Cumulative_Return_Plus_One'].
 ↪cummax()
spxt_monthly['Drawdown'] = spxt_monthly['Cumulative_Return_Plus_One'] /␣
 ↪spxt_monthly['Rolling_Max'] - 1
spxt_monthly.drop(columns=['Cumulative_Return_Plus_One', 'Rolling_Max'],␣
 ↪inplace=True)

# Calculate cumulative returns and drawdown for treas_10y
```

```
treas_10y_monthly['Cumulative_Return'] = (1 +␣
↪treas_10y_monthly['Monthly_Return']).cumprod() - 1
treas_10y_monthly['Cumulative_Return_Plus_One'] = 1 +␣
↪treas_10y_monthly['Cumulative_Return']
treas_10y_monthly['Rolling_Max'] =␣
↪treas_10y_monthly['Cumulative_Return_Plus_One'].cummax()
treas_10y_monthly['Drawdown'] = treas_10y_monthly['Cumulative_Return_Plus_One']␣
↪/ treas_10y_monthly['Rolling_Max'] - 1
treas_10y_monthly.drop(columns=['Cumulative_Return_Plus_One', 'Rolling_Max'],␣
↪inplace=True)

# Convert to DataFrame
portfolio_monthly = strategy[['Date', 'strategy_return']].copy().
↪set_index('Date')
portfolio_monthly = portfolio_monthly.rename(columns={'strategy_return':␣
↪'Portfolio_Monthly_Return'})

# Calculate cumulative returns and drawdown for the portfolio
portfolio_monthly['Portfolio_Cumulative_Return'] = (1 +␣
↪portfolio_monthly['Portfolio_Monthly_Return']).cumprod() - 1
portfolio_monthly['Portfolio_Cumulative_Return_Plus_One'] = 1 +␣
↪portfolio_monthly['Portfolio_Cumulative_Return']
portfolio_monthly['Portfolio_Rolling_Max'] =␣
↪portfolio_monthly['Portfolio_Cumulative_Return_Plus_One'].cummax()
portfolio_monthly['Portfolio_Drawdown'] =␣
↪portfolio_monthly['Portfolio_Cumulative_Return_Plus_One'] /␣
↪portfolio_monthly['Portfolio_Rolling_Max'] - 1
portfolio_monthly.drop(columns=['Portfolio_Cumulative_Return_Plus_One',␣
↪'Portfolio_Rolling_Max'], inplace=True)

# Merge "spxt_monthly" and "treas_10y_monthly" into "portfolio_monthly" to␣
↪compare cumulative returns
portfolio_monthly = portfolio_monthly.join(
    spxt_monthly['Monthly_Return'].rename('SPXT_Monthly_Return'),
    how='left'
).join(
    spxt_monthly['Cumulative_Return'].rename('SPXT_Cumulative_Return'),
    how='left'
).join(
    spxt_monthly['Drawdown'].rename('SPXT_Drawdown'),
    how='left'
).join(
    treas_10y_monthly['Monthly_Return'].rename('10Y_Monthly_Return'),
    how='left'
).join(
    treas_10y_monthly['Cumulative_Return'].rename('10Y_Cumulative_Return'),
```

```
    how='left'
).join(
    treas_10y_monthly['Drawdown'].rename('10Y_Drawdown'),
    how='left'
)
```

[44]:
```
display(portfolio_monthly)
```

```
            Portfolio_Monthly_Return  Portfolio_Cumulative_Return  \
Date
1990-01-31                  -0.06713                     -0.06713
1990-02-28                   0.01288                     -0.05511
1990-03-31                   0.02650                     -0.03007
1990-04-30                  -0.02497                     -0.05429
1990-05-31                   0.09752                      0.03793
…                                  …                            …
2025-08-31                   0.02027                     42.28506
2025-09-30                   0.00675                     42.57706
2025-10-31                   0.00685                     42.87548
2025-11-30                   0.01020                     43.32308
2025-12-31                  -0.00662                     43.02959

            Portfolio_Drawdown  SPXT_Monthly_Return  SPXT_Cumulative_Return  \
Date
1990-01-31             0.00000             -0.06713                -0.06713
1990-02-28             0.00000              0.01288                -0.05511
1990-03-31             0.00000              0.02650                -0.03007
1990-04-30            -0.02497             -0.02497                -0.05429
1990-05-31             0.00000              0.09752                 0.03793
…                            …                    …                       …
2025-08-31             0.00000              0.02027                36.70243
2025-09-30             0.00000              0.03650                38.07857
2025-10-31             0.00000              0.02341                38.99354
2025-11-30             0.00000              0.00245                39.09156
2025-12-31            -0.00662              0.00061                39.11610

            SPXT_Drawdown  10Y_Monthly_Return  10Y_Cumulative_Return  \
Date
1990-01-31        0.00000            -0.01987               -0.01987
1990-02-28        0.00000            -0.00023               -0.02010
1990-03-31        0.00000            -0.00001               -0.02011
1990-04-30       -0.02497            -0.01411               -0.03394
1990-05-31        0.00000             0.03148               -0.00353
…                     …                   …                      …
2025-08-31        0.00000             0.01668                5.41258
2025-09-30        0.00000             0.00675                5.45584
2025-10-31        0.00000             0.00685                5.50005
2025-11-30        0.00000             0.01020                5.56636
```

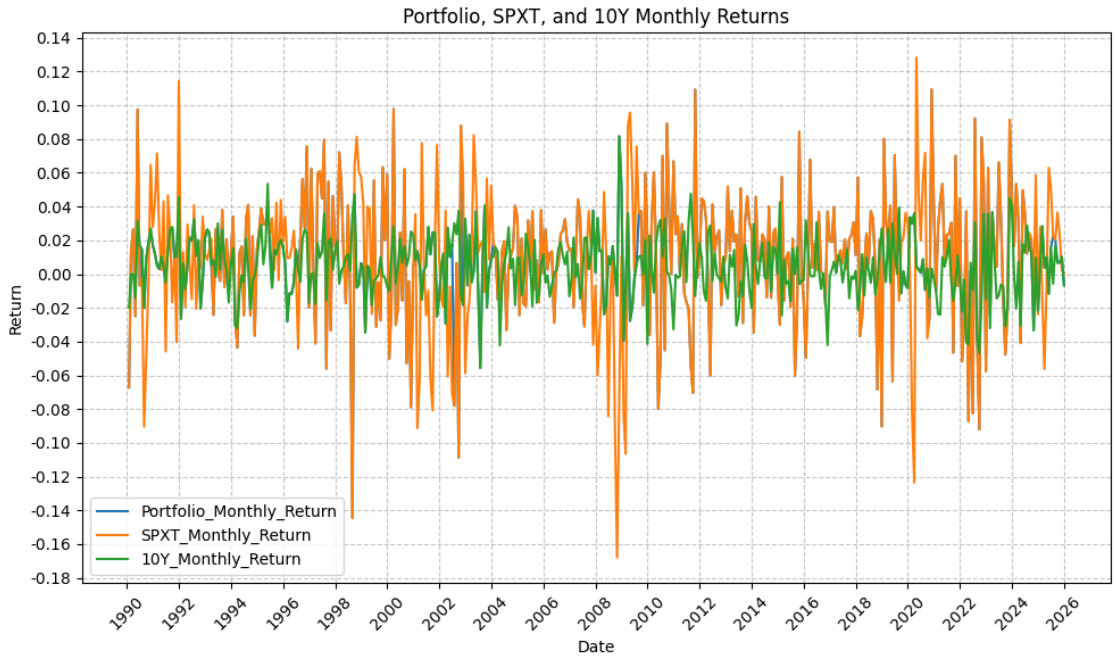
```
2025-12-31        0.00000            -0.00662                 5.52288

            10Y_Drawdown
Date
1990-01-31       0.00000
1990-02-28      -0.00023
1990-03-31      -0.00024
1990-04-30      -0.01436
1990-05-31       0.00000
…                   …
2025-08-31      -0.11316
2025-09-30      -0.10717
2025-10-31      -0.10106
2025-11-30      -0.09189
2025-12-31      -0.09790

[432 rows x 9 columns]
```

Next, we'll look at performance for the assets and portfolio.

0.8 Performance Statistics

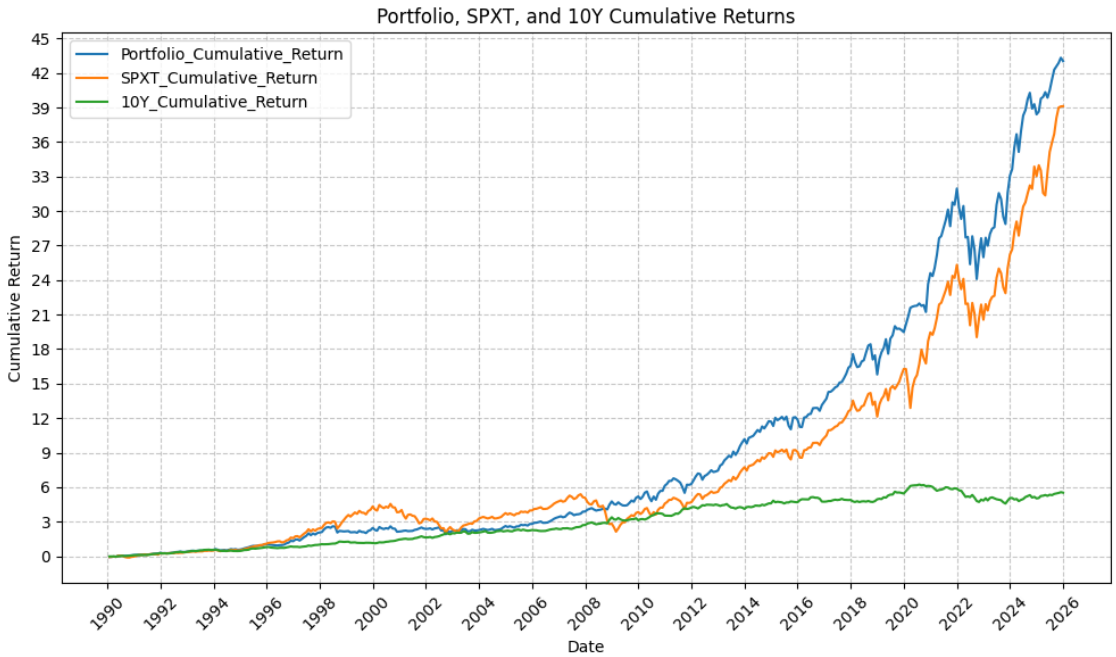
We can then plot the monthly returns:

[45]:
```
plot_timeseries(
    price_df=portfolio_monthly,
    plot_start_date=start_date,
    plot_end_date=end_date,
    plot_columns=["Portfolio_Monthly_Return", "SPXT_Monthly_Return",␣
 ↪"10Y_Monthly_Return"],
    title="Portfolio, SPXT, and 10Y Monthly Returns",
    x_label="Date",
    x_format="Year",
    x_tick_spacing=2,
    x_tick_rotation=45,
    y_label="Return",
    y_format="Decimal",
    y_format_decimal_places=2,
    y_tick_spacing=0.02,
    grid=True,
    legend=True,
    export_plot=False,
    plot_file_name=None,
)
```



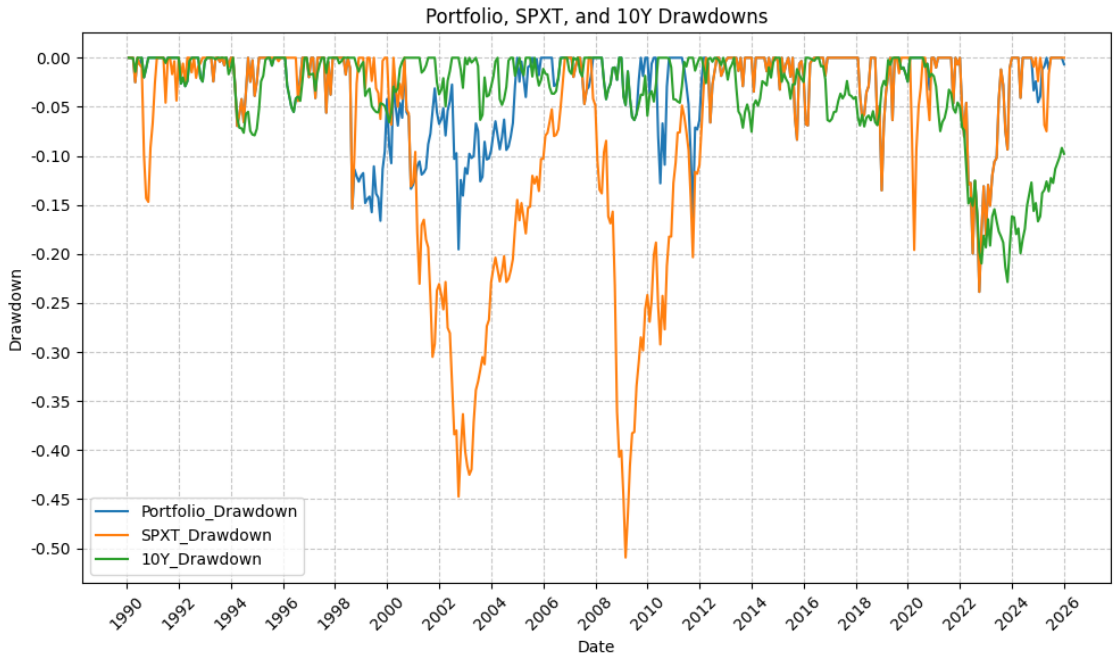
And cumulative returns:

[46]:
```
plot_timeseries(
    price_df=portfolio_monthly,
    plot_start_date=start_date,
    plot_end_date=end_date,
    plot_columns=["Portfolio_Cumulative_Return", "SPXT_Cumulative_Return",␣
↪"10Y_Cumulative_Return"],
    title="Portfolio, SPXT, and 10Y Cumulative Returns",
    x_label="Date",
    x_format="Year",
    x_tick_spacing=2,
    x_tick_rotation=45,
    y_label="Cumulative Return",
    y_format="Decimal",
    y_format_decimal_places=0,
    y_tick_spacing=3,
    grid=True,
    legend=True,
    export_plot=False,
    plot_file_name=None,
)
```



And drawdowns:

[47]:
```
plot_timeseries(
    price_df=portfolio_monthly,
    plot_start_date=start_date,
    plot_end_date=end_date,
    plot_columns=["Portfolio_Drawdown", "SPXT_Drawdown", "10Y_Drawdown"],
    title="Portfolio, SPXT, and 10Y Drawdowns",
    x_label="Date",
    x_format="Year",
    x_tick_spacing=2,
    x_tick_rotation=45,
    y_label="Drawdown",
    y_format="Decimal",
    y_format_decimal_places=2,
    y_tick_spacing=0.05,
    grid=True,
    legend=True,
    export_plot=False,
    plot_file_name=None,
)
```



Finally, we can run the summary statistics on the strategy portfolio, SPY, and TLT with the following code:

[48]:
```
port_sum_stats = summary_stats(
    fund_list=["Portfolio", "SPXT", "10Y"],
    df=portfolio_monthly[["Portfolio_Monthly_Return"]],
    period="Monthly",
    use_calendar_days=False,
    excel_export=False,
    pickle_export=False,
    output_confirmation=False,
)

spy_sum_stats = summary_stats(
    fund_list=["Portfolio", "SPXT", "10Y"],
    df=portfolio_monthly[["SPXT_Monthly_Return"]],
    period="Monthly",
    use_calendar_days=False,
    excel_export=False,
    pickle_export=False,
    output_confirmation=False,
)

tlt_sum_stats = summary_stats(
    fund_list=["Portfolio", "SPXT", "10Y"],
    df=portfolio_monthly[["10Y_Monthly_Return"]],
```

```
    period="Monthly",
    use_calendar_days=False,
    excel_export=False,
    pickle_export=False,
    output_confirmation=False,
)

sum_stats = pd.concat([port_sum_stats, spy_sum_stats, tlt_sum_stats])
sum_stats
```

[48]:

```
                          Annual Mean Return (Arithmetic)  \
Portfolio_Monthly_Return                          0.11267
SPXT_Monthly_Return                               0.11392
10Y_Monthly_Return                                0.05420

                          Annualized Volatility  Annualized Sharpe Ratio  \
Portfolio_Monthly_Return                0.11897                  0.94701
SPXT_Monthly_Return                     0.14735                  0.77311
10Y_Monthly_Return                      0.06331                  0.85606

                          CAGR (Geometric)  Monthly Max Return  \
Portfolio_Monthly_Return           0.11086             0.10946
SPXT_Monthly_Return                0.10799             0.12819
10Y_Monthly_Return                 0.05347             0.08169

                         Monthly Max Return (Date)  Monthly Min Return  \
Portfolio_Monthly_Return                2020-11-30            -0.14458
SPXT_Monthly_Return                     2020-04-30            -0.16795
10Y_Monthly_Return                      2008-11-30            -0.05558

                         Monthly Min Return (Date)  Max Drawdown       Peak  \
Portfolio_Monthly_Return                1998-08-31      -0.23867 2021-12-31
SPXT_Monthly_Return                     2008-10-31      -0.50949 2007-10-31
10Y_Monthly_Return                      2003-07-31      -0.22865 2020-07-31

                            Trough Recovery Date  Days to Recovery  MAR Ratio
Portfolio_Monthly_Return 2022-09-30    2023-12-31         457.00000    0.46449
SPXT_Monthly_Return      2009-02-28    2012-03-31        1127.00000    0.21196
10Y_Monthly_Return       2023-10-31           NaT               NaN    0.23386
```

Based on the above, our strategy portfolio outperforms both stocks and bonds, with lower drawdowns.

0.9 Conclusions

This was a interesting exercise to evaluate the performance of different asset classes during Fed tightening and easing cycles. The results are not particularly surprising, but it is interesting to see the data and plots to confirm the economic intuition that stocks perform well during tightening

cycles (economic strength) and bonds perform well during easing cycles (economic weakness). The results are certainly dependent on the specific time period or regime, and also on the assumption made for how to handle the periods of neutral policy (i.e. no change in FFR).

0.10 Future Investigation

A couple of ideas sound intriguing for future investigation:

- Does a commodity index (such as GSCI) exhibit differing behavior than gold?
- How does leverage affect the returns that are observed for the strategy portfolio, stocks, and bonds?
- Do other Fed tightening/loosening cycles exhibit the same behavior for returns?

0.11 References

1. https://fred.stlouisfed.org/series/DFEDTARU
2. https://fred.stlouisfed.org/series/DFEDTARL
3. https://fred.stlouisfed.org/series/DFEDTAR